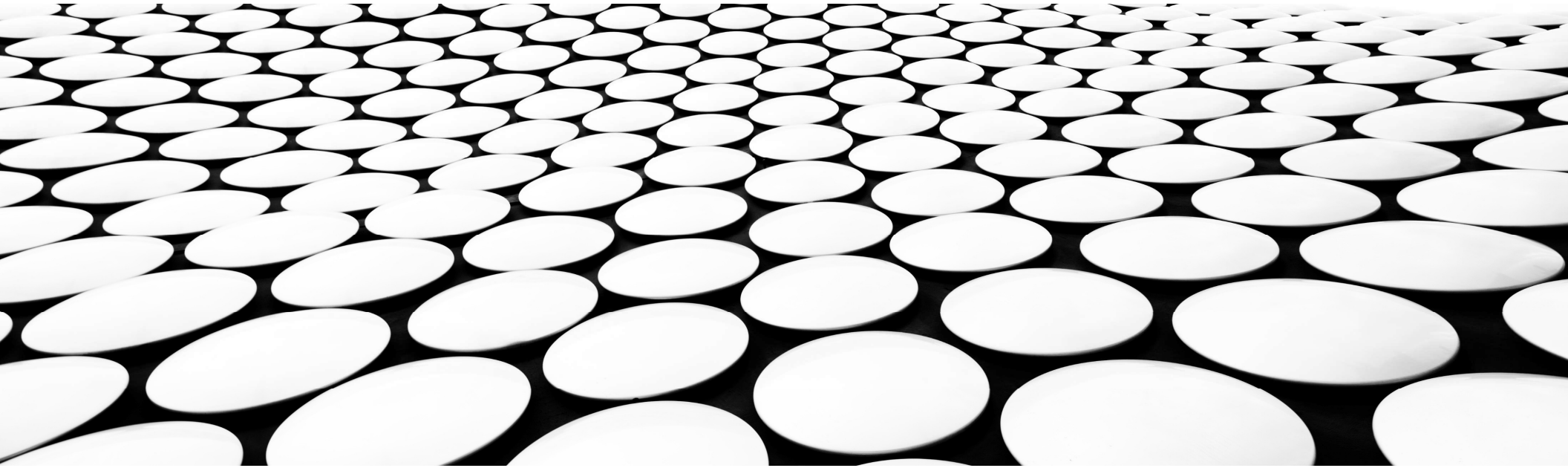
Unsupervised learning

Clustering, Auto-encoders

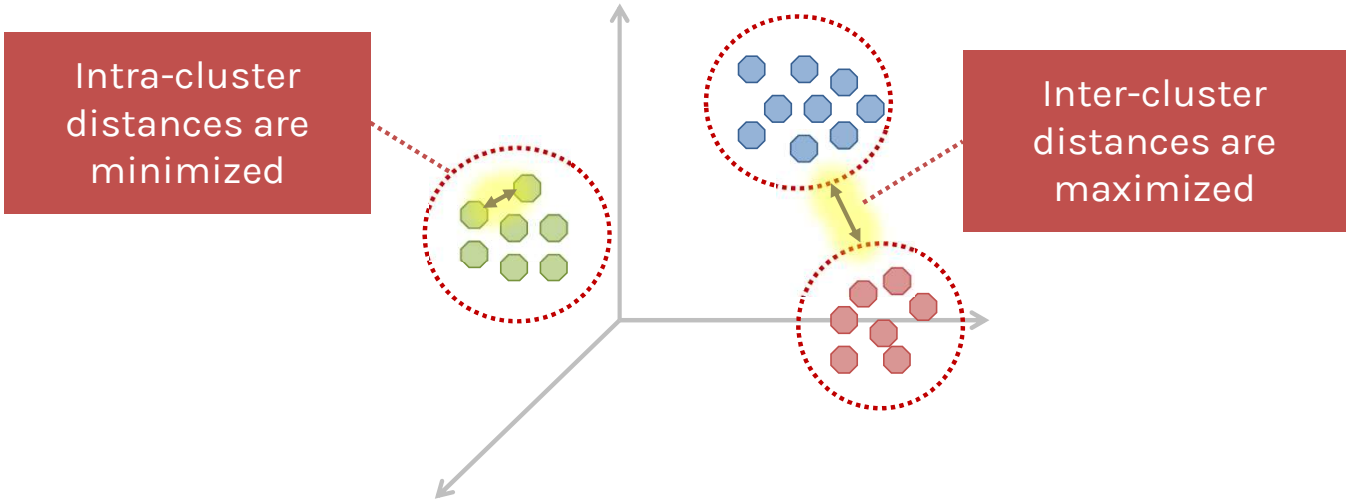


Outline

- Clustering
  - Partitioning: k-Means algorithm
  - Hierarchical: Agglomerative clustering
  - Density-based: DBSCAN
- Cluster evaluation
- Autoencoders

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups

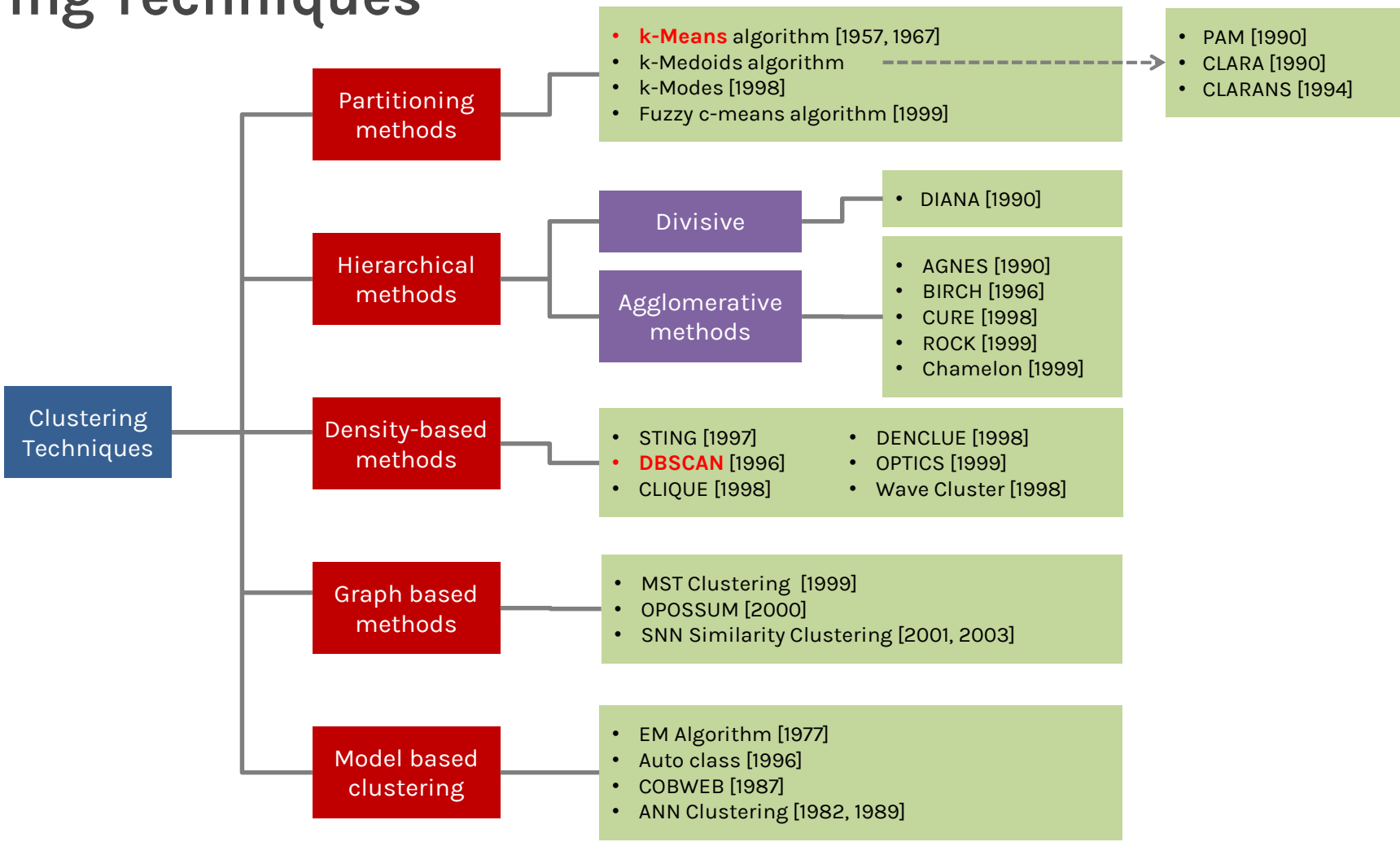


Applications of Cluster Analysis

- **Understanding**
  - Group related documents for browsing
  - Group users with similar preferences for recommendations
  - Group genes and proteins that have similar functionality
  - Group stocks with similar price fluctuations
- **Summarization**
  - Reduce the size of large data sets

	Discovered Clusters	Industry Group
1	Applied-Matl-DOWN,Bay-Network-Down,3-COM-DOWN, Cabletron-Sys-DOWN,CISCO-DOWN,HP-DOWN, DSC-Comm-DOWN,INTEL-DOWN,LSI-Logic-DOWN, Micron-Tech-DOWN,Texas-Inst-Down,Tellabs-Inc-Down, Natl-Semiconduct-DOWN,Oracl-DOWN,SGI-DOWN, Sun-DOWN	Technology1-DOWN
2	Apple-Comp-DOWN,Autodesk-DOWN,DEC-DOWN, ADV-Micro-Device-DOWN,Andrew-Corp-DOWN, Computer-Assoc-DOWN,Circuit-City-DOWN, Compaq-DOWN, EMC-Corp-DOWN, Gen-Inst-DOWN, Motorola-DOWN,Microsoft-DOWN,Scientific-Atl-DOWN	Technology2-DOWN
3	Fannie-Mae-DOWN,Fed-Home-Loan-DOWN, MBNA-Corp-DOWN,Morgan-Stanley-DOWN	Financial-DOWN
4	Baker-Hughes-UP,Dresser-Inds-UP,Halliburton-HLD-UP, Louisiana-Land-UP,Phillips-Petro-UP,Unocal-UP, Schlumberger-UP	Oil-UP

Clustering Techniques



Notion of a Cluster can be Ambiguous





6

How many clusters?

2



4

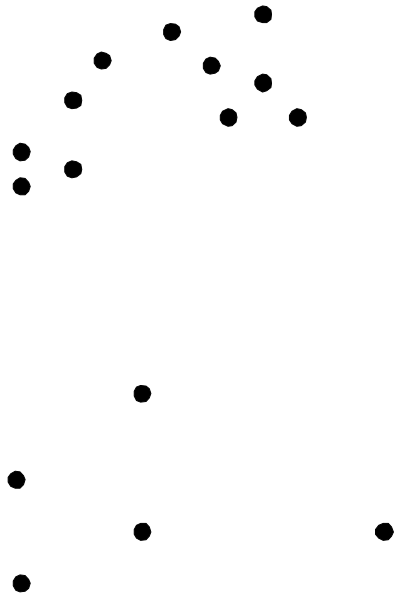


Types of Clusterings

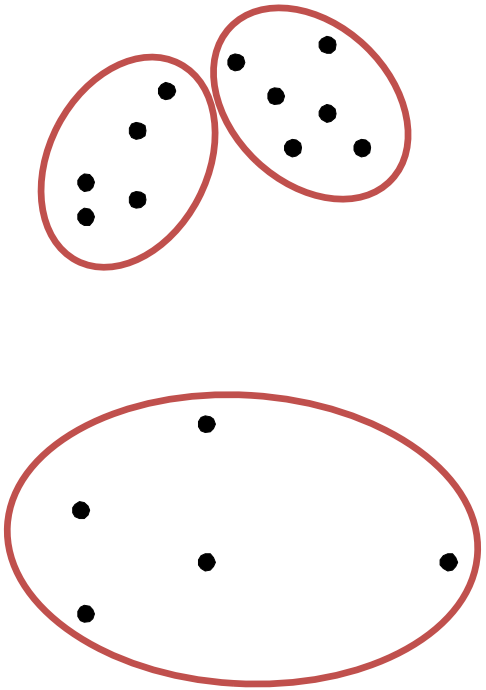
- A **clustering** is a set of clusters

- Important distinction between **hierarchical** and **partitional** sets of clusters
  - **Partitional Clustering**
    - A division of data objects into **non-overlapping** subsets (clusters) such that each data object belongs in exactly one subset
  - **Hierarchical clustering**
    - A set of nested clusters organized as a hierarchical tree

Partitional Clustering

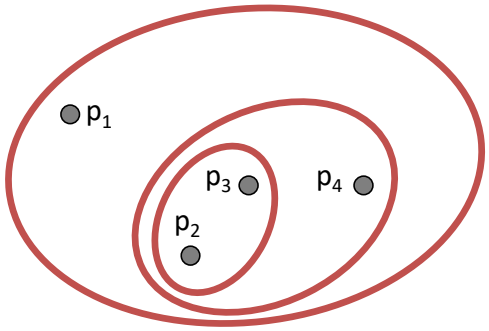


Original Points

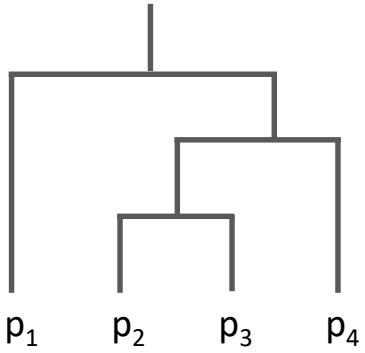


A Partitional Clustering

Hierarchical Clustering



Traditional Hierarchical Clustering



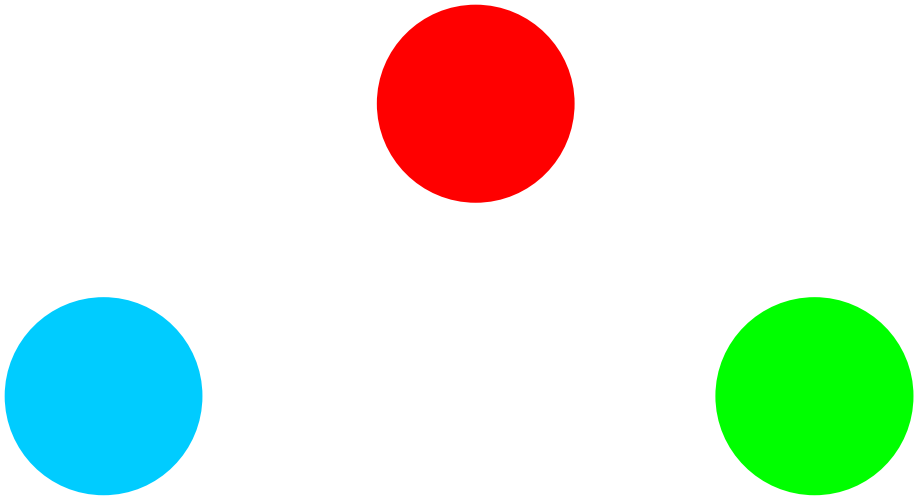
Traditional Dendrogram

Types of Clusters

- Well-separated clusters
- Prototype-based clusters
- Contiguity-based clusters
- Density-based clusters
- Described by an Objective Function

Types of Clusters: Well-Separated

- Well-Separated Clusters:
  - A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters

Types of Clusters: Prototype-Based

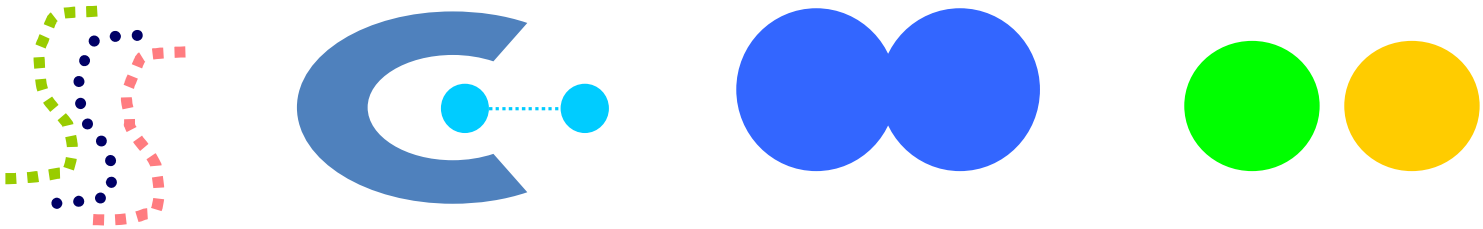
- Prototype-based
  - A cluster is a set of objects such that an object in a cluster is closer (more similar) to the prototype or "center" of a cluster, than to the center of any other cluster
  - The center of a cluster is often a **centroid**, the average of all the points in the cluster, or a **medoid**, the most "representative" point of a cluster



4 center-based clusters

Types of Clusters: Contiguity-Based

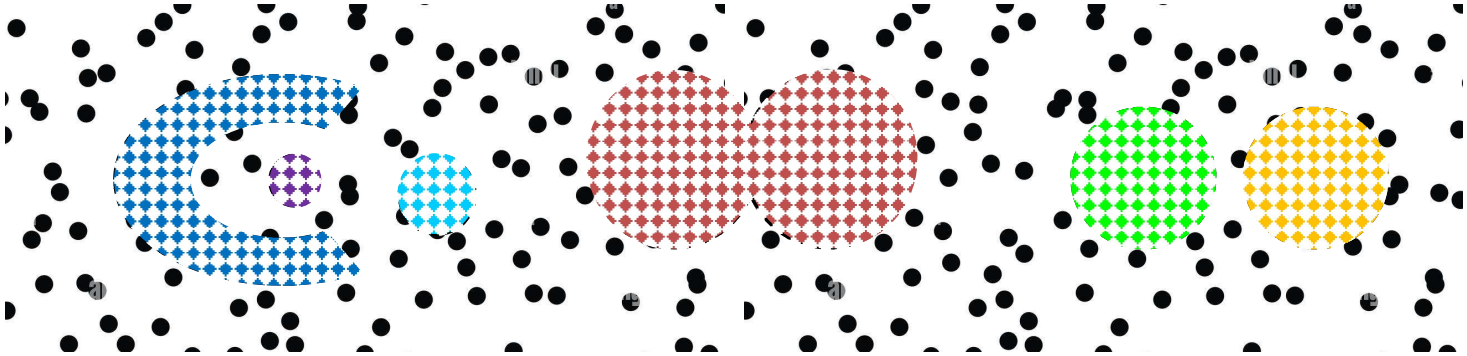
- Contiguous Cluster (Nearest neighbor or Transitive)
  - A cluster is a set of points such that a point in a cluster is closer (or more similar) to one or more other points in the cluster than to any point not in the cluster.



8 contiguous clusters

Types of Clusters: Density-Based

- Density-based
  - A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.
  - Used when the clusters are irregular or intertwined, and when noise and outliers are present.



6 density-based clusters

Clustering Algorithms

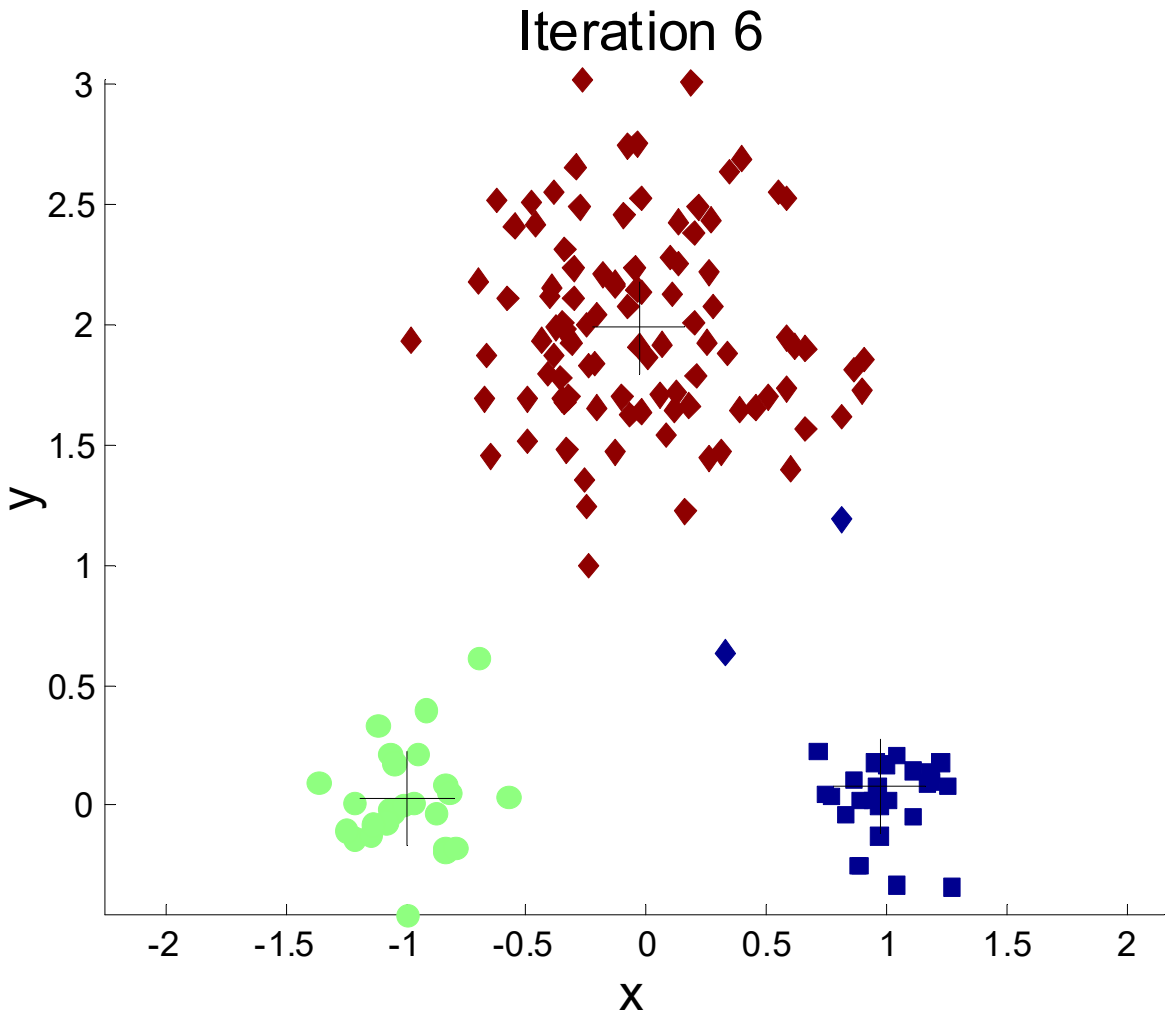
- K-means and its variants
- Hierarchical clustering
- Density-based clustering

K-means Clustering

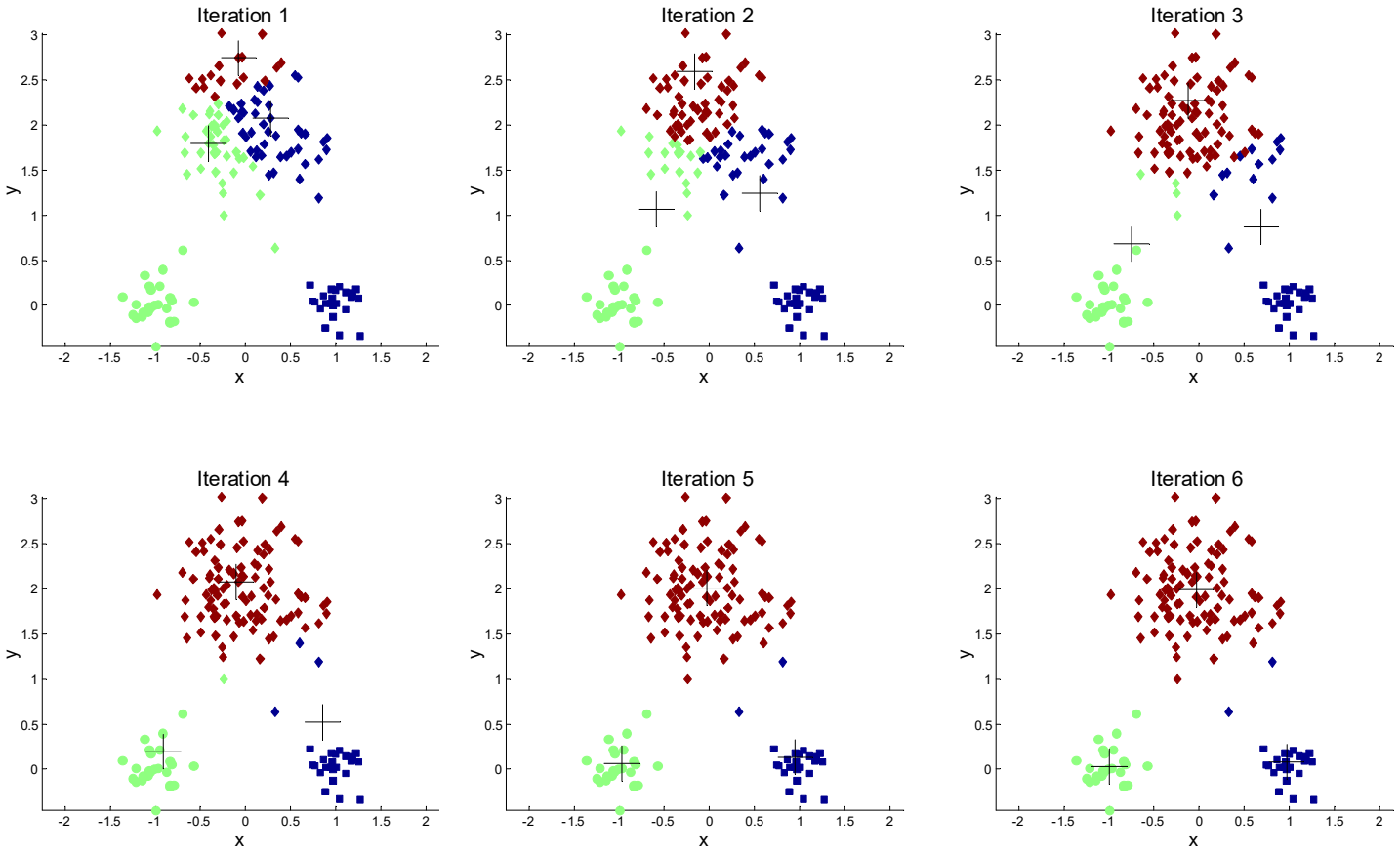
- Partitional clustering approach
- **Number of clusters**, **K**, must be specified
- Each cluster is associated with a **centroid** (center point)
- Each point is assigned to the cluster with the closest centroid
- The basic algorithm is very simple:

1: Select K points as the initial centroids.
2: **repeat**
3: Form K clusters by assigning all points to the closest centroid.
4: Recompute the centroid of each cluster.
5: **until** The centroids don't change

Example of K-means Clustering



Example of K-means Clustering



K-means Clustering – Details

- Simple iterative algorithm
  - Choose initial centroids; **repeat** {assign each point to a nearest centroid; re-compute cluster centroids} **until centroids stop changing**.
- Initial centroids are often chosen randomly.
  - Clusters produced can vary from one run to another
- The centroid is (typically) the mean of the points in the cluster, but other definitions are possible.
- K-means will converge for common proximity measures with appropriately defined centroid
- Most of the convergence happens in the first few iterations.
  - Often the stopping condition is changed to '**Until relatively few points change clusters**'
- Complexity is O(n * K * I * d)
  - n = number of points, K = number of clusters,
    I = number of iterations, d = number of attributes

K-means Objective Function

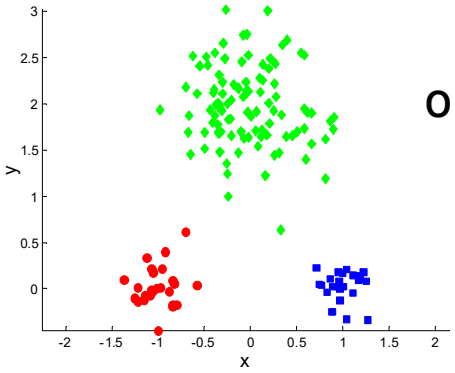
- A common objective function (used with Euclidean distance measure) is **Sum of Squared Error** (**SSE**)
  - For each point, the error is the distance to the nearest cluster center
  - To get SSE, we square these errors and sum them.

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

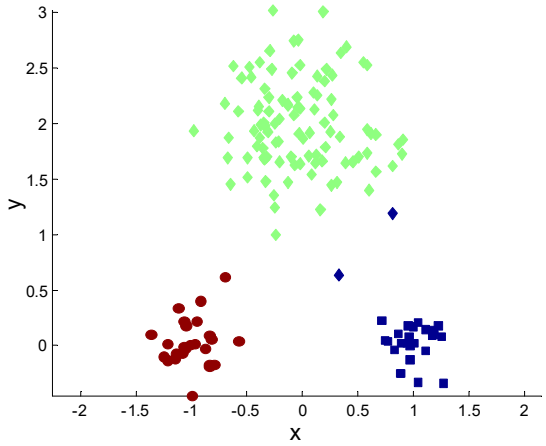
  - x is a data point in cluster C_i and m_i is the centroid (mean) for cluster C_i

- SSE improves in each iteration of K-means until it reaches a local or global minima.

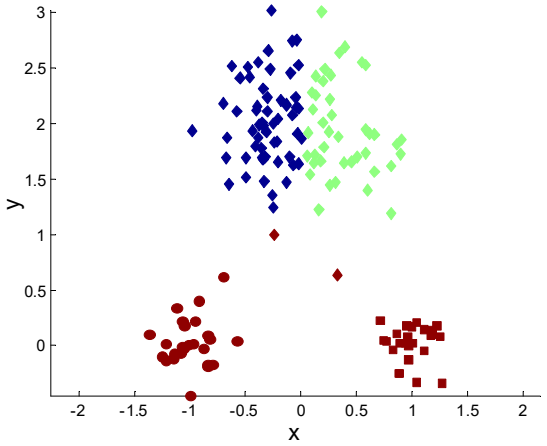
Two different K-means Clusterings



Original Points

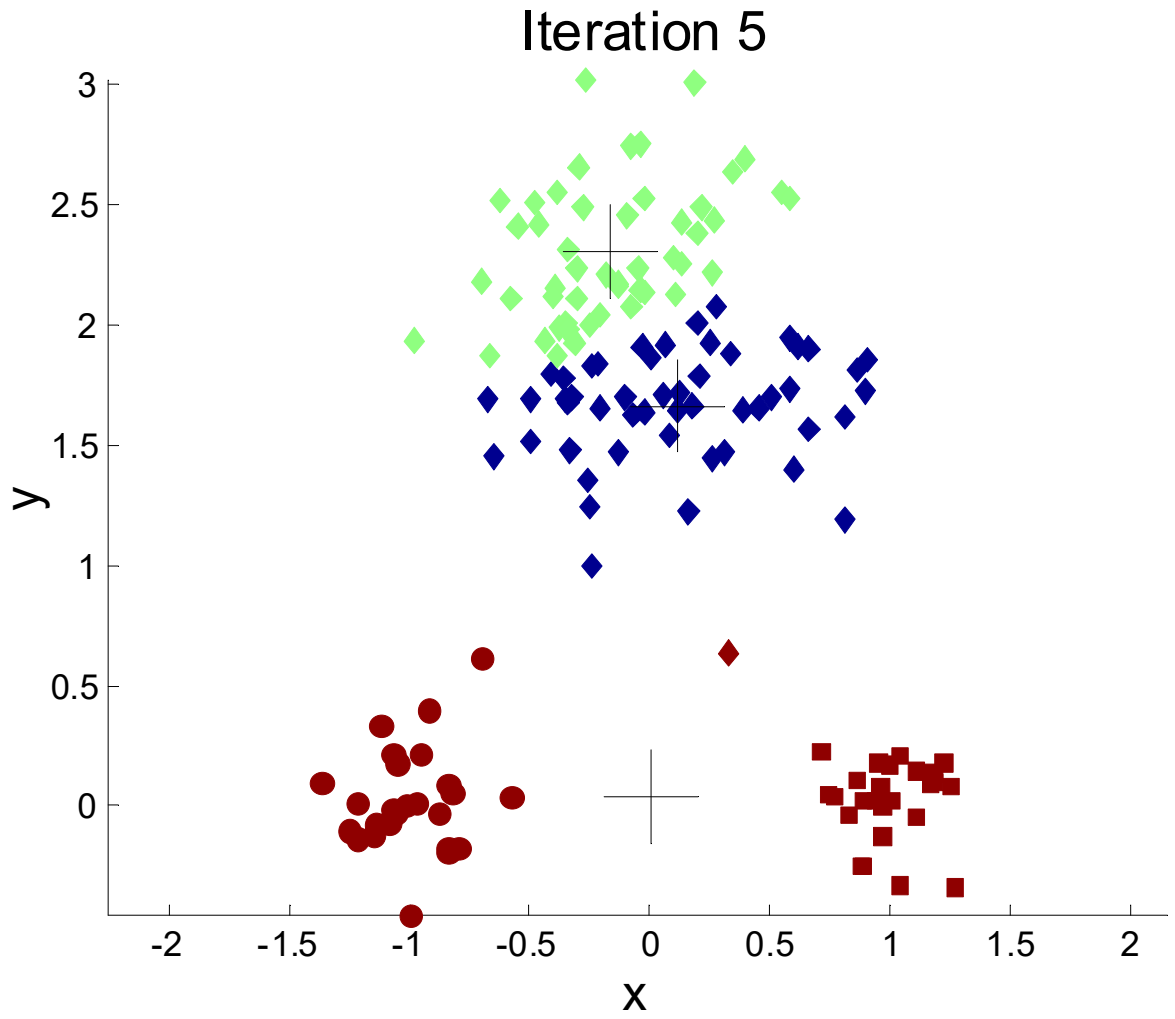


Optimal Clustering

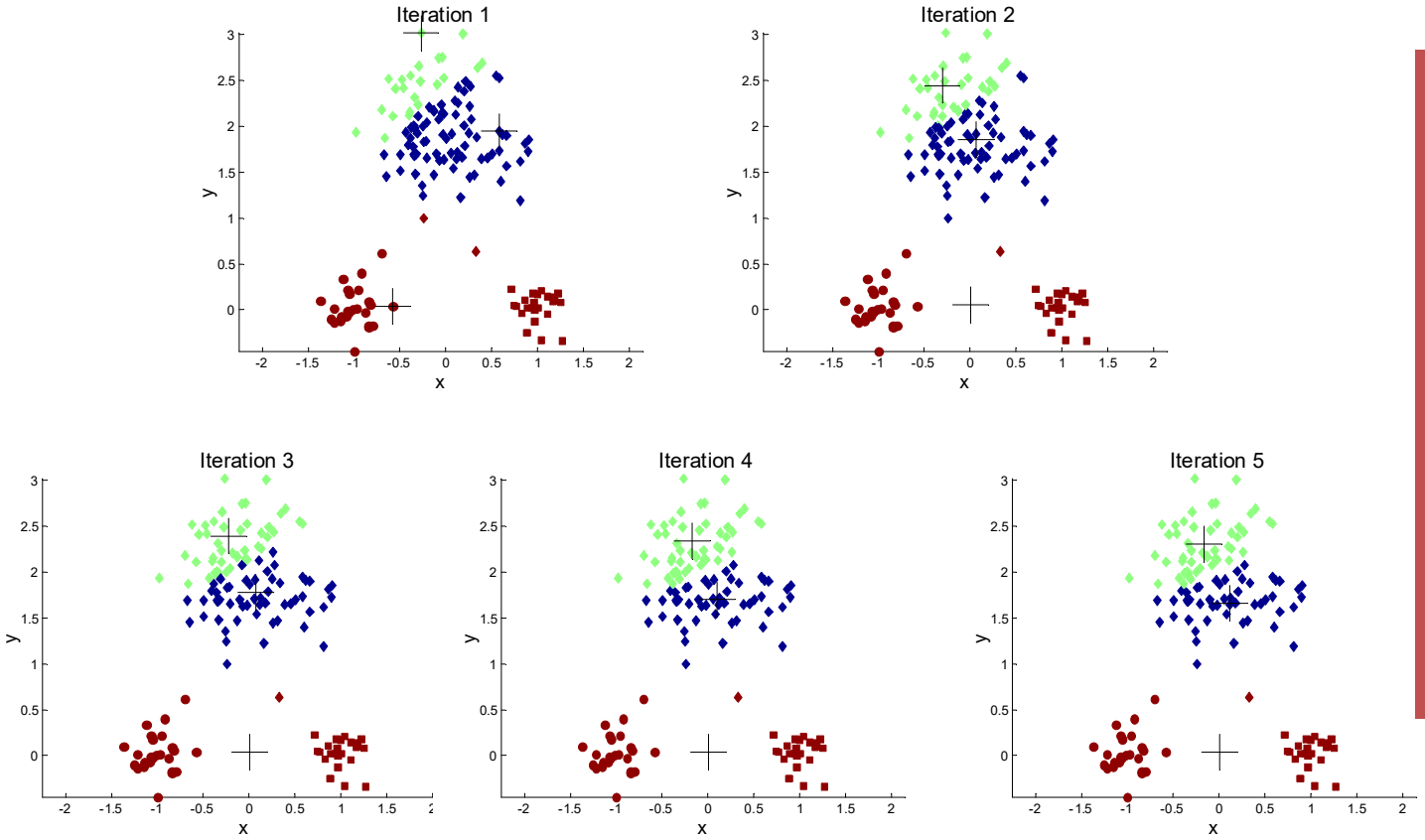


Sub-optimal Clustering

Importance of Choosing Initial Centroids …



Importance of Choosing Initial Centroids …



Depending on the choice of initial centroids, some clusters may get merged or remain separate

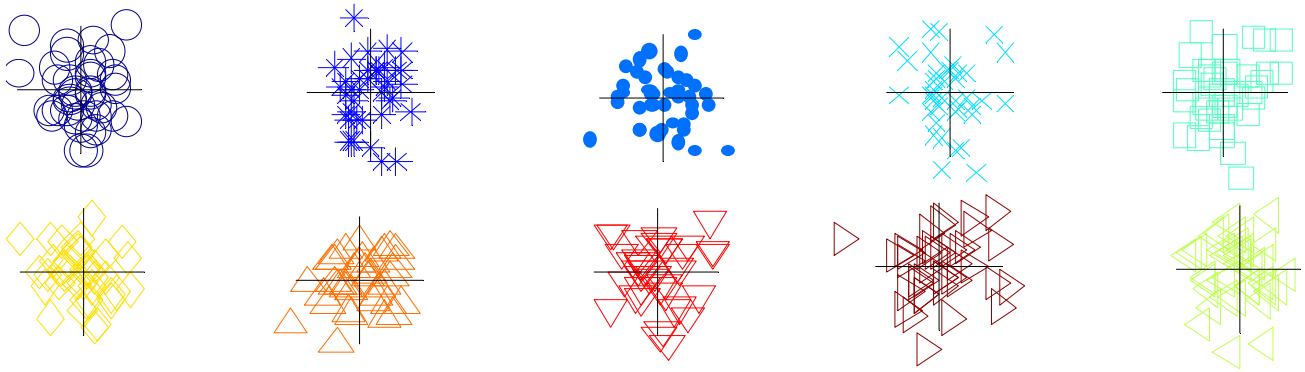
Problems with Selecting Initial Points

- If there are *K* 'real' clusters, then the chance of selecting one centroid from each cluster is small.
  - Chance is relatively small when *K* is large
  - If clusters are the same size, *n*, then

$$P = \frac{\text{number of ways to select one centroid from each cluster}}{\text{number of ways to select } K \text{ centroids}} = \frac{K!n^K}{(Kn)^K} = \frac{K!}{K^K}$$

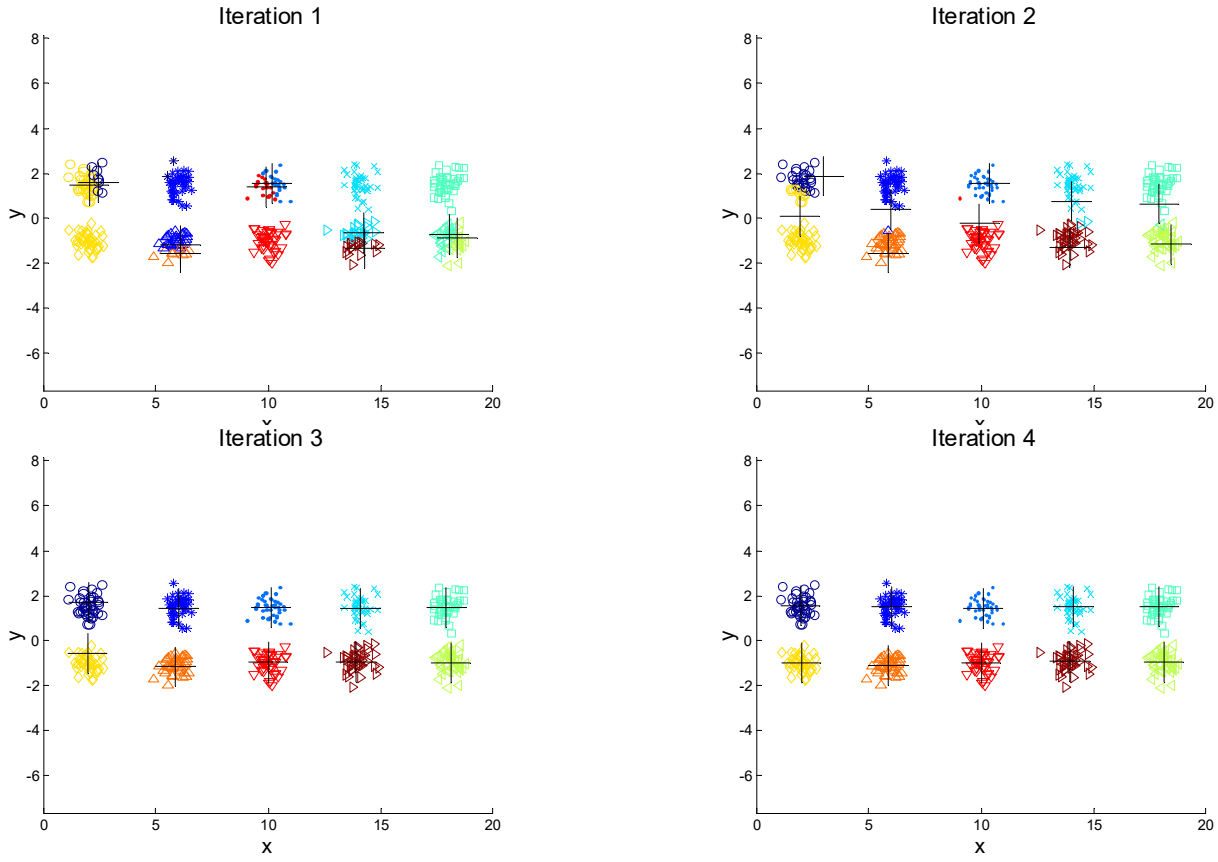
  - For example, if *K* = 10, then probability = $10!/10^{10}$ = 0.00036
  - Sometimes the initial centroids will readjust themselves in 'right' way, and sometimes they don't
  - Consider an example of five pairs of clusters

10 Clusters Example



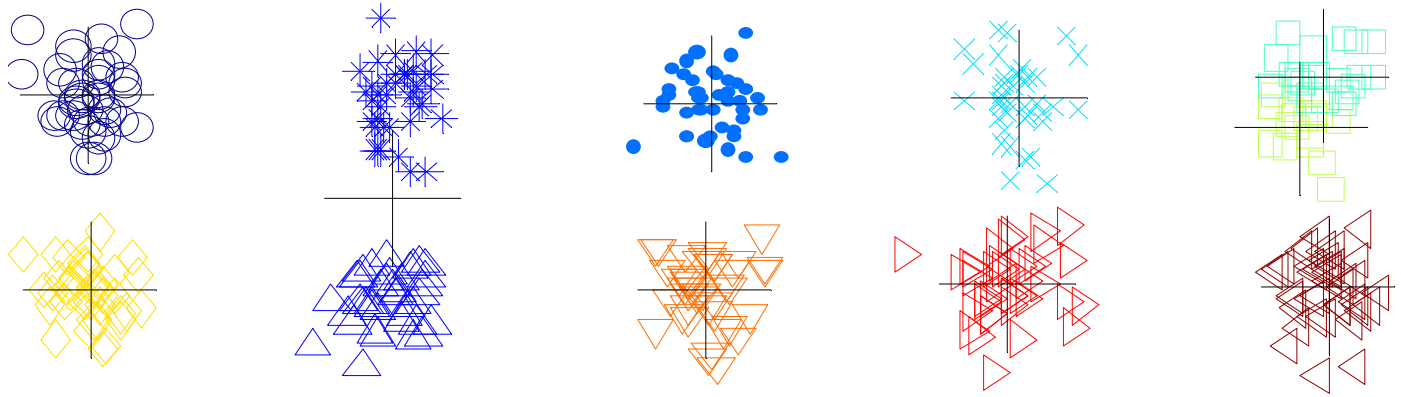
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



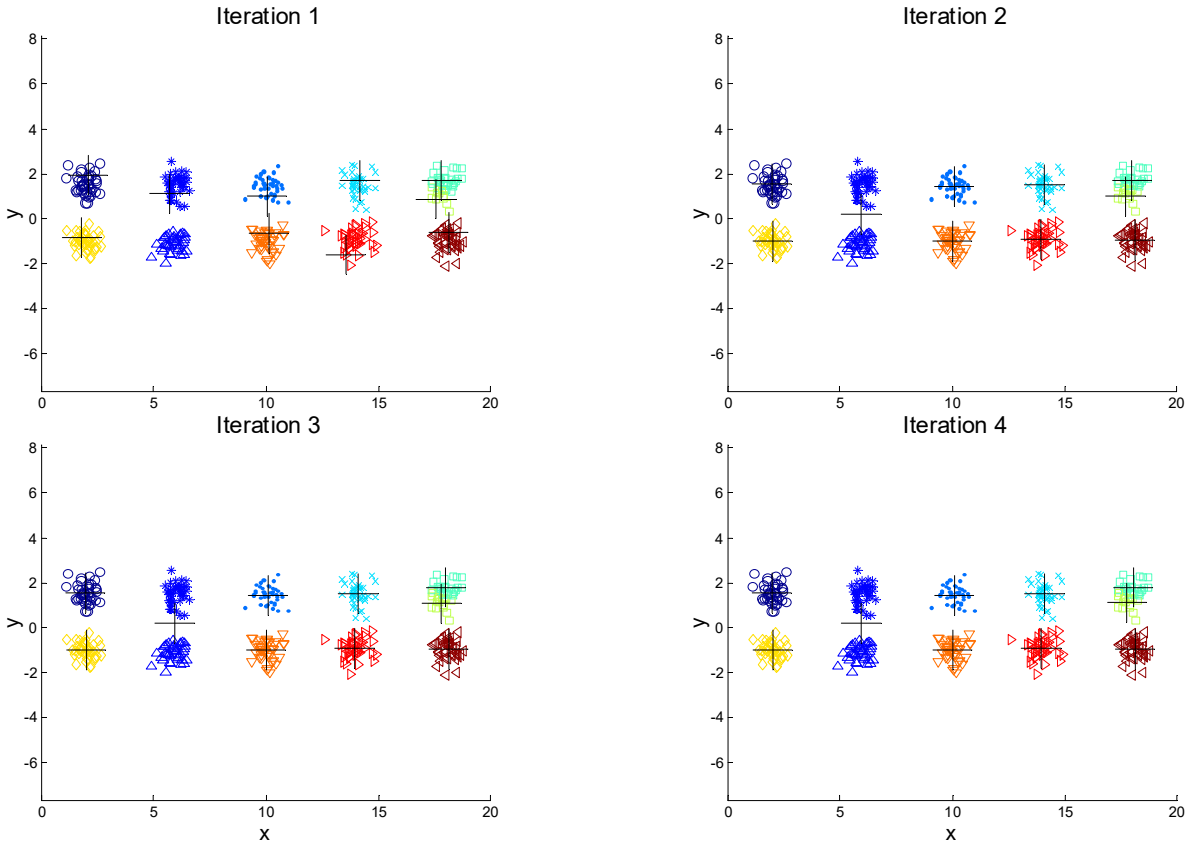
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one

Solutions to Initial Centroids Problem

- **Multiple runs**
  - Helps, but probability is not on your side
- **Use some strategy to select the k initial centroids** and then select among these initial centroids
  - Select most widely separated
    - **K-means++** is a robust way of doing this selection
  - Use hierarchical clustering to determine initial centroids

K-means++

- This approach can be slower than random initialization, but very consistently produces better results in terms of SSE
- To select a set of initial centroids, C, perform the following:

---

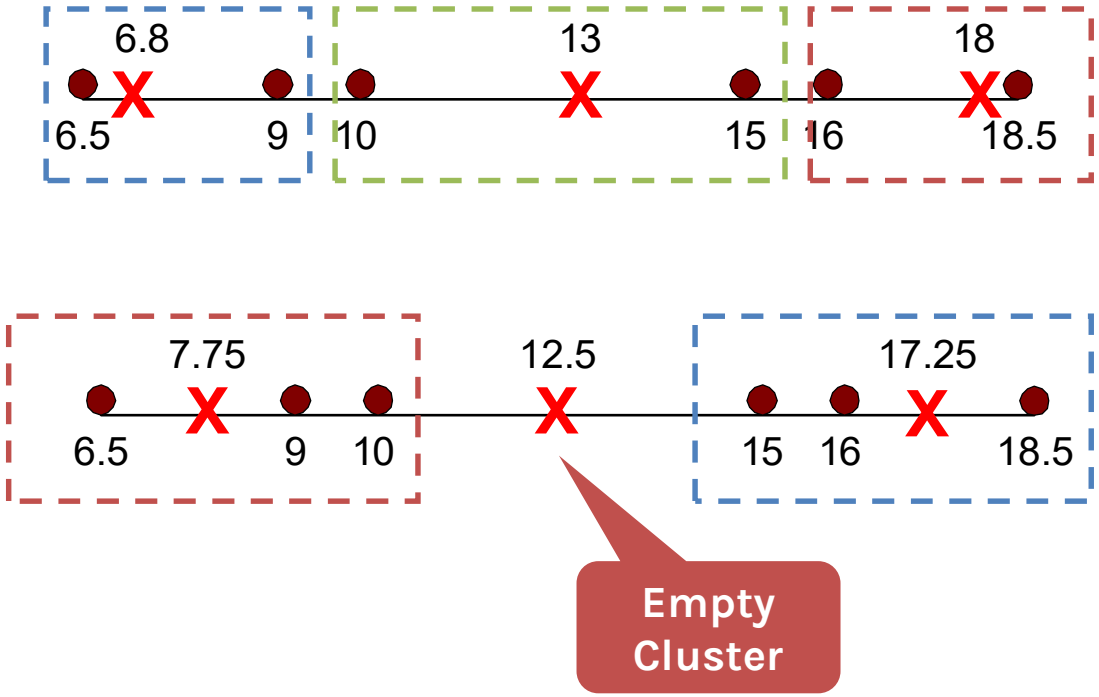
1. select an initial point at random to be the first centroid
2. **for** k – 1 steps
3. **for each** of the N points, x_i, $1 \leq i \leq N$, find the minimum squared distance to the currently selected centroids, C_1, …, C_j, $1 \leq j < k$, i.e., $\min_j d^2(C_j, x_i)$
4. randomly select a new centroid by choosing a point with probability proportional to $\frac{\min_j d^2(C_j, x_i)}{\sum_i \min_j d^2(C_j, x_i)}$
5. **end-for**

Pre-processing and Post-processing

- **Pre-processing**
  - Normalize the data
  - Eliminate outliers
- **Post-processing**
  - Eliminate empty clusters and small clusters that may represent outliers
  - Split ‘loose’ clusters, i.e., clusters with relatively high SSE
  - Merge clusters that are ‘close’ and that have relatively low SSE
  - These steps can be used multiple times during the clustering process
    - ISODATA

Empty Clusters

- K-means can yield empty clusters



Handling Empty Clusters

- Basic K-means algorithm can yield **empty clusters**
- Several strategies:
  - Choose the point that contributes most to SSE, and make it a new centroid
  - Choose a point from the cluster with the highest SSE, and make it a new centroid
  - If there are several empty clusters, the above can be repeated several times.

Updating Centers Incrementally

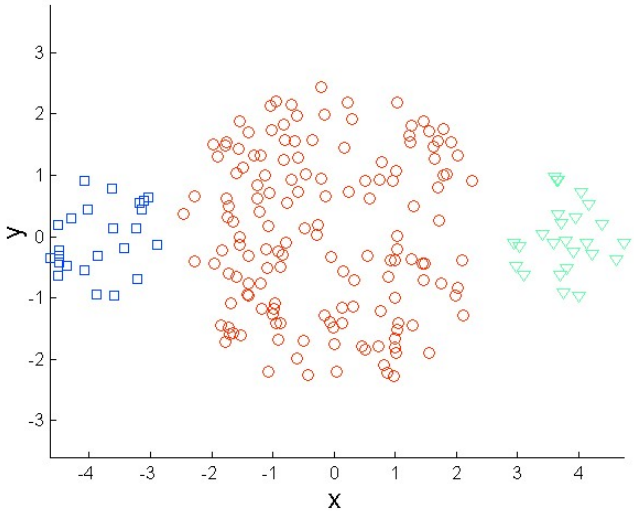
- In the basic K-means algorithm, centroids are updated after all points are assigned to a centroid
- An alternative is to update the centroids after each assignment (incremental approach)
  - Each assignment updates zero or two centroids
  - More expensive
  - Introduces an order dependency
  - Never get an empty cluster

Limitations of K-means

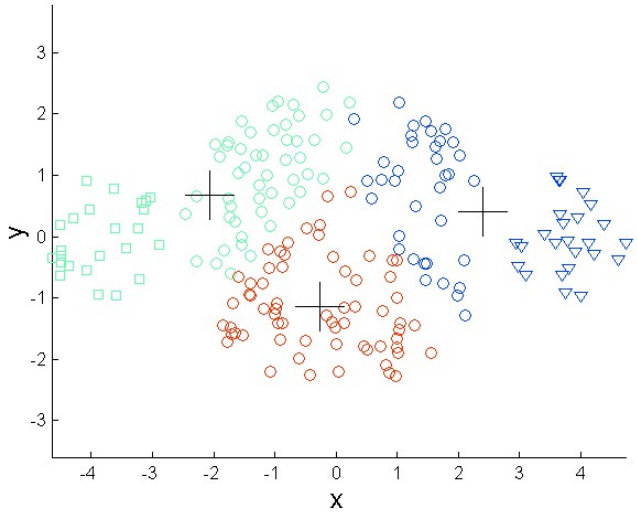
- K-means has problems when clusters are of differing
  - **Sizes**
  - **Densities**
  - **Non-globular shapes** (non-convex)

- K-means has problems when the data contains **outliers**.

Limitations of K-means: **Differing Sizes**

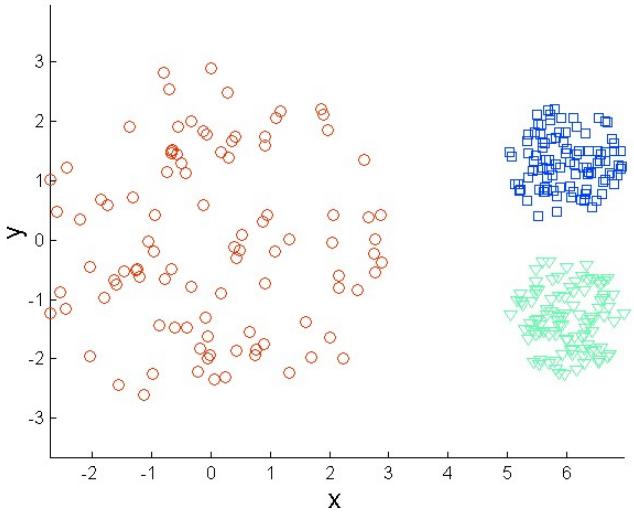


Original Points

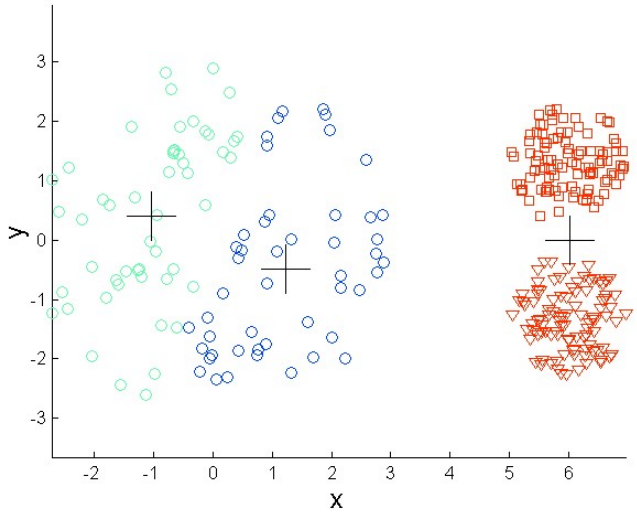


K-means (3 Clusters)

Limitations of K-means: **Differing Density**

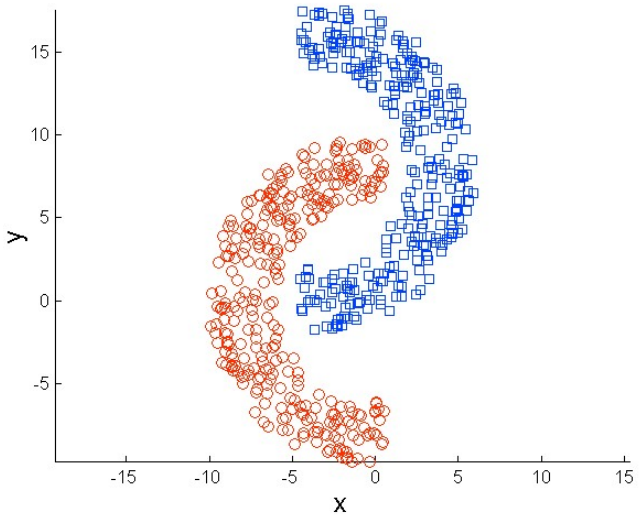


Original Points

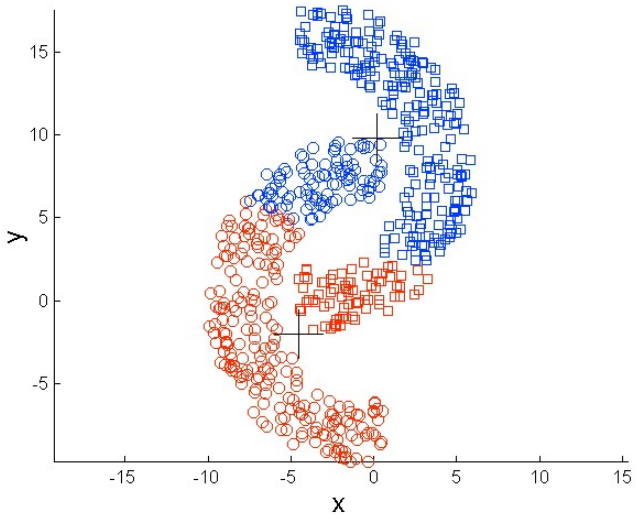


K-means (3 Clusters)

Limitations of K-means: **Non-globular Shapes (non-convex)**

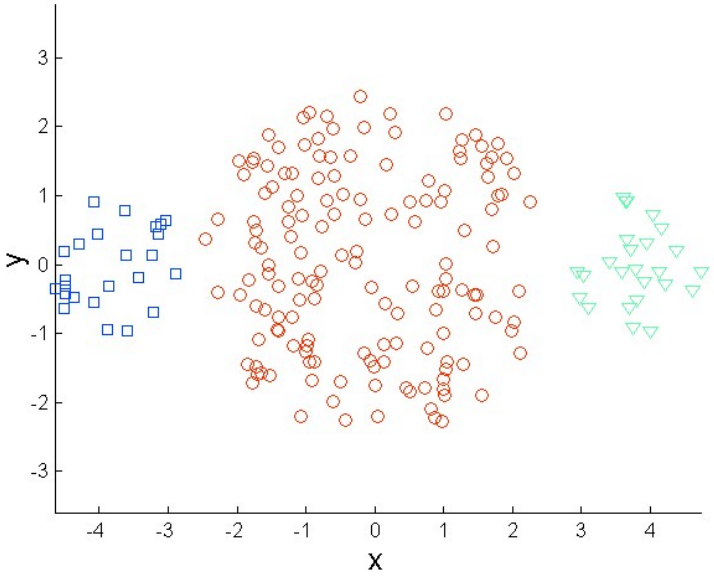


Original Points

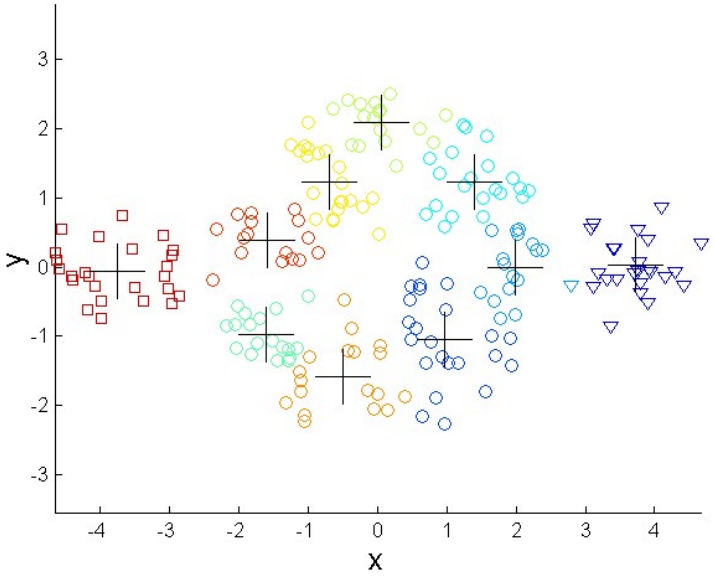


K-means (2 Clusters)

Overcoming K-means Limitations



Original Points

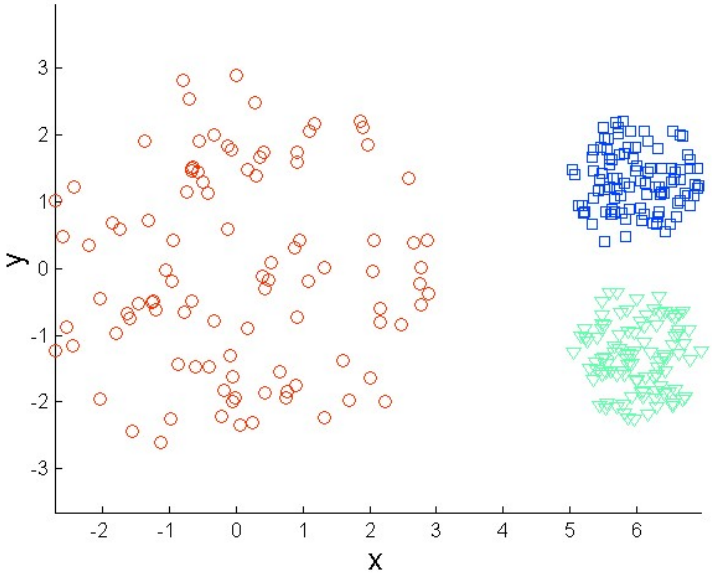


K-means Clusters

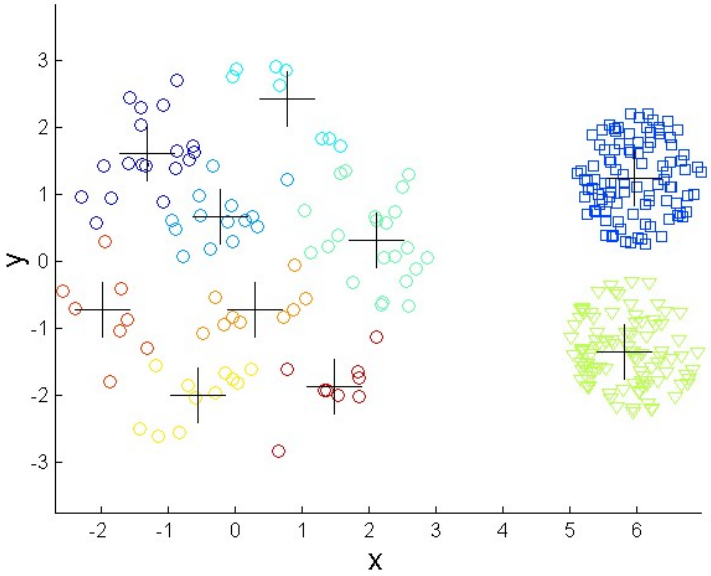
One solution is to use many clusters.

Find parts of clusters, but need to put together.

Overcoming K-means Limitations



Original Points

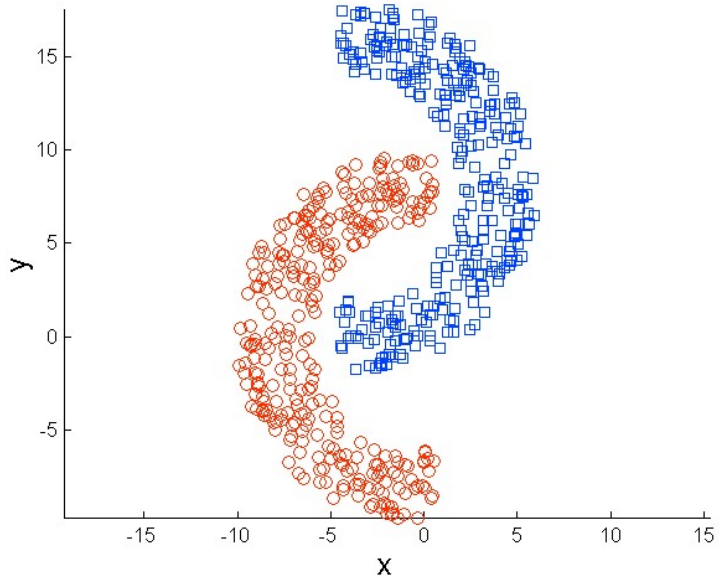


K-means Clusters

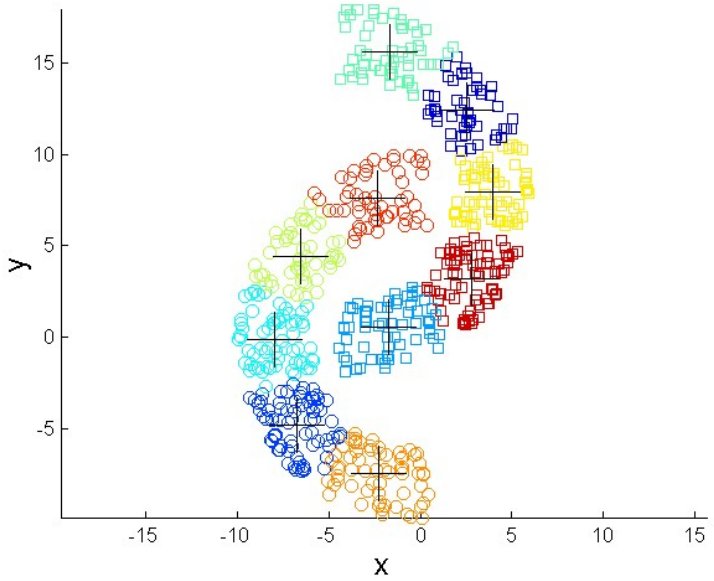
One solution is to use many clusters.

Find parts of clusters, but need to put together.

Overcoming K-means Limitations



Original Points



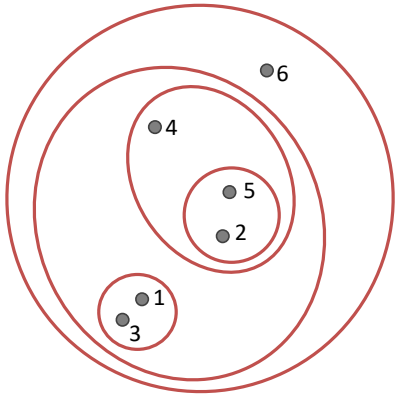
K-means Clusters

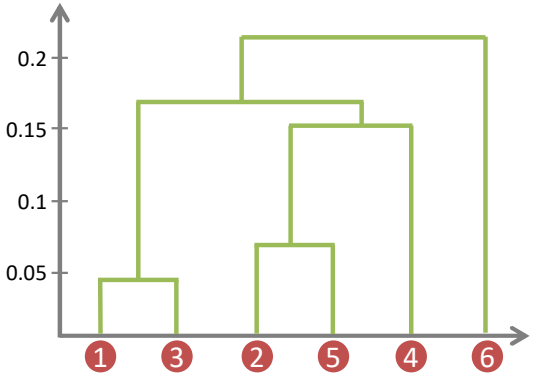
One solution is to use many clusters.

Find parts of clusters, but need to put together.

Hierarchical Clustering

- Produces a set of nested clusters organized as a hierarchical tree
- Can be visualized as a dendrogram
  - A tree like diagram that records the sequences of merges or splits





Strengths of Hierarchical Clustering

- Do not have to assume any **particular number of clusters**
  - Any desired number of clusters can be obtained by ‘cutting’ the dendrogram at the proper level

- They may correspond to meaningful taxonomies
  - Example in biological sciences (e.g., animal kingdom, phylogeny reconstruction, …)

Hierarchical Clustering

- Two main types of hierarchical clustering:
  - **Agglomerative**:
    - Start with the points as individual clusters
    - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left
  - **Divisive**:
    - Start with one, all-inclusive cluster
    - At each step, split a cluster until each cluster contains an individual point (or there are k clusters)

- Traditional hierarchical algorithms use a similarity or distance matrix
  - Merge or split one cluster at a time

Agglomerative Clustering Algorithm

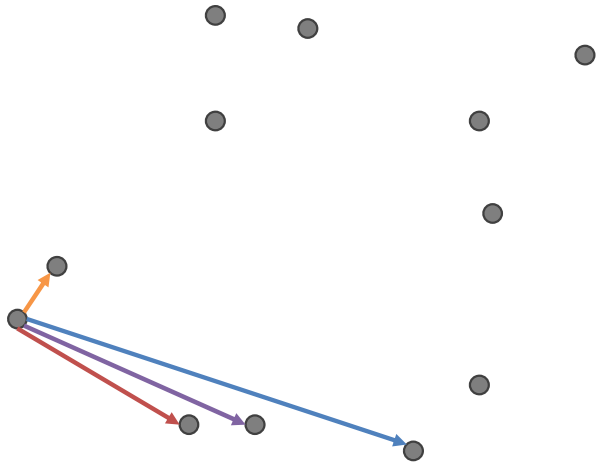
- Most popular hierarchical clustering technique
  - **Key Idea: Successively merge closest clusters**
- Basic algorithm is straightforward:

1. Compute the proximity matrix
2. Let each data point be a cluster
3. **Repeat**
4. Merge the two closest clusters
5. Update the proximity matrix
6. **Until** only a single cluster remains

- Key operation is the computation of the **proximity of two clusters**
  - Different approaches to defining the distance between clusters distinguish the different algorithms

Starting Situation

- Start with clusters of individual points and a proximity matrix

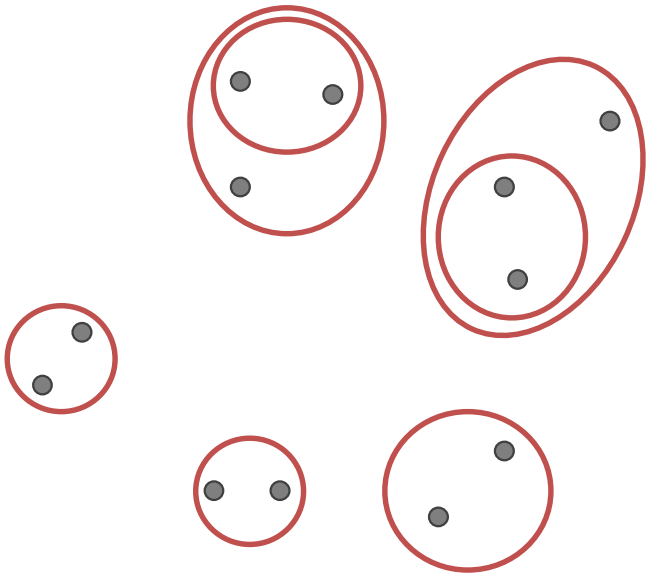


	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

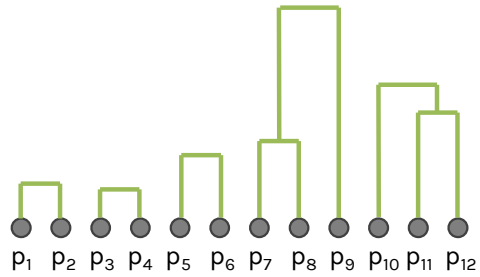
Starting Situation

- Start with clusters of individual points and a proximity matrix



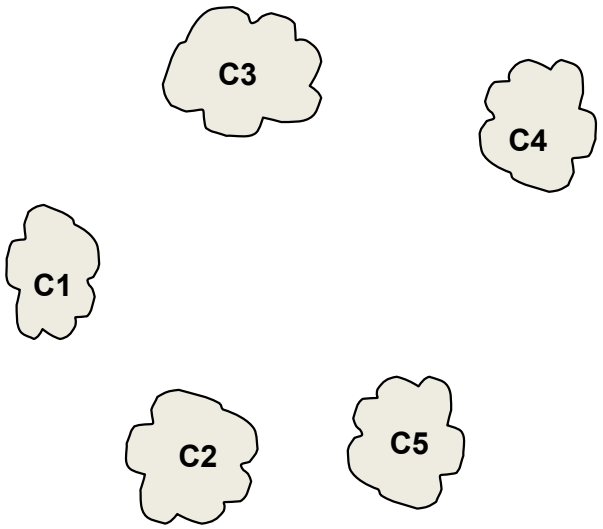
	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix



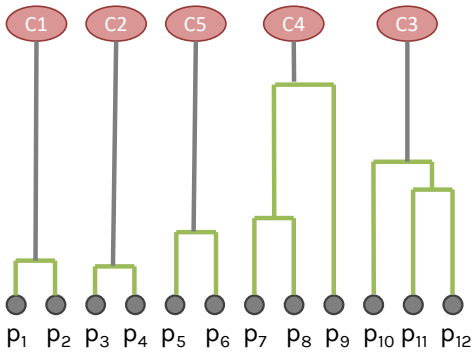
Intermediate Situation

- After some merging steps, we have some clusters



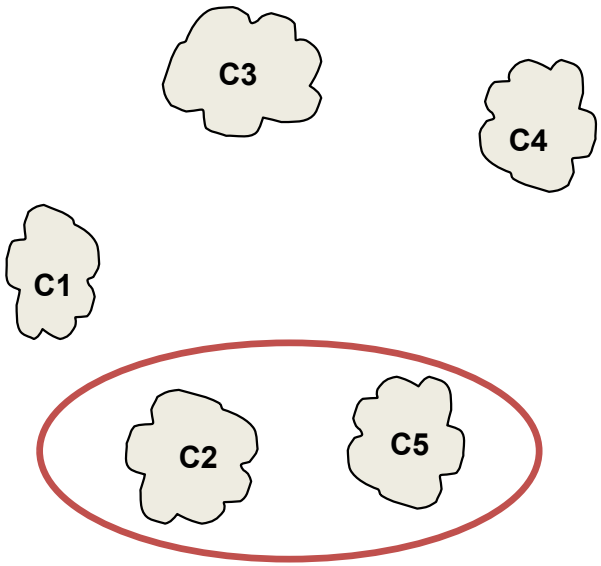
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



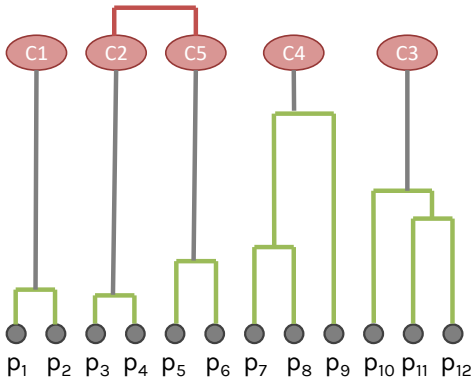
Intermediate Situation

- We want to merge the two closest clusters (C2 and C5) and update the proximity matrix.



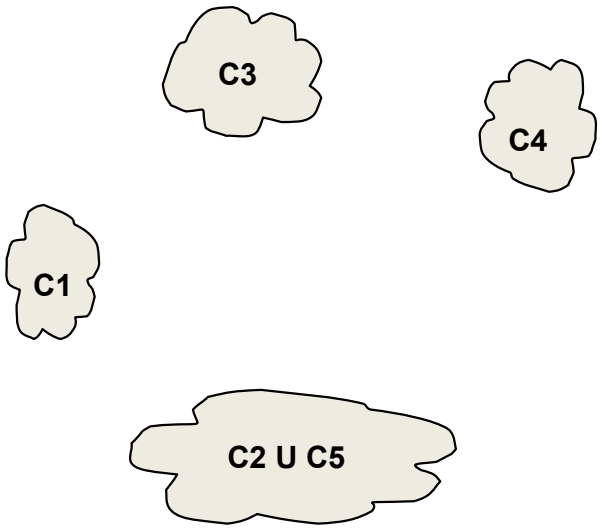
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



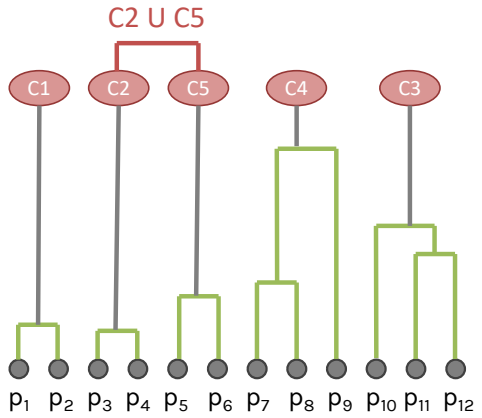
After Merging

- The question is “How do we update the proximity matrix?”

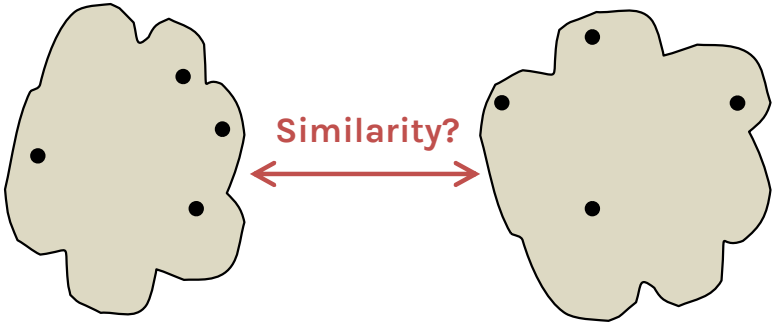


	C1	C2 U C5	C3	C4
C1		?		
C2 U C5	?	?	?	?
C3		?		
C4		?		

Proximity Matrix



How to Define Inter-Cluster Distance

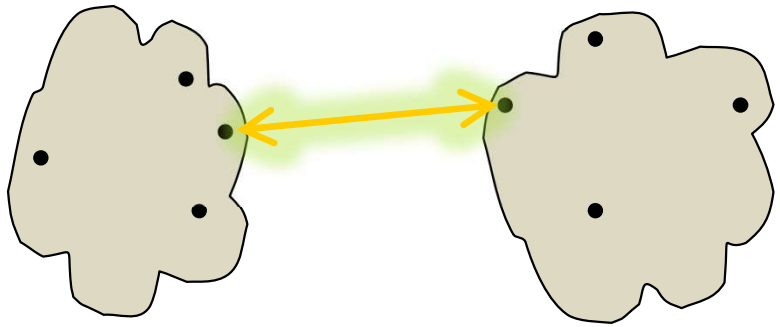


- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity

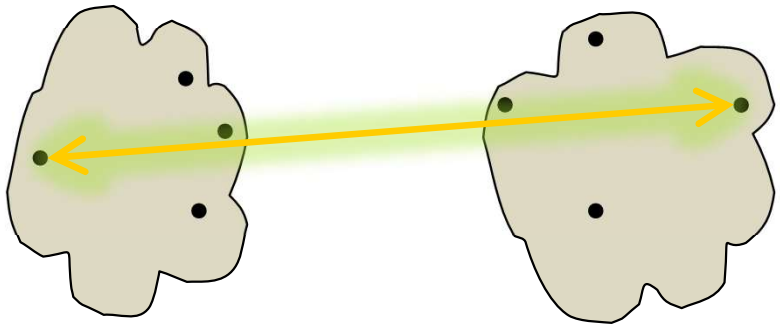


- **MIN**
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity

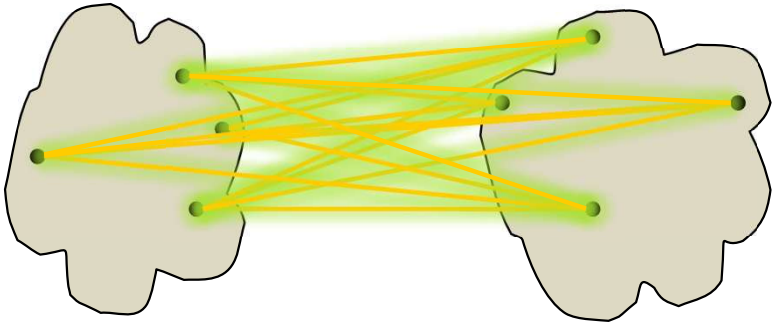


- MIN
- **MAX**
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity

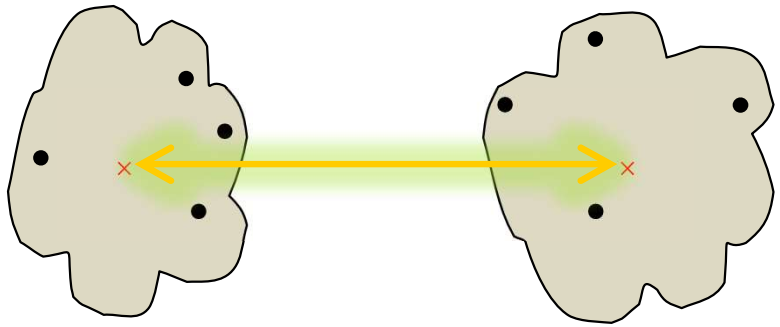


- MIN
- MAX
- **Group Average**
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity



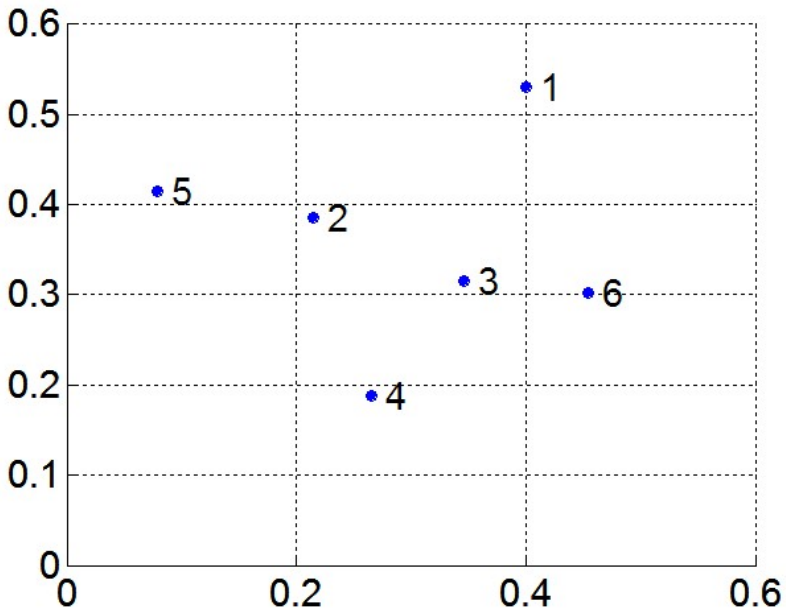
- MIN
- MAX
- Group Average
- **Distance Between Centroids**
- Other methods driven by an objective function
  - Ward's Method uses squared error

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

MIN or Single Link

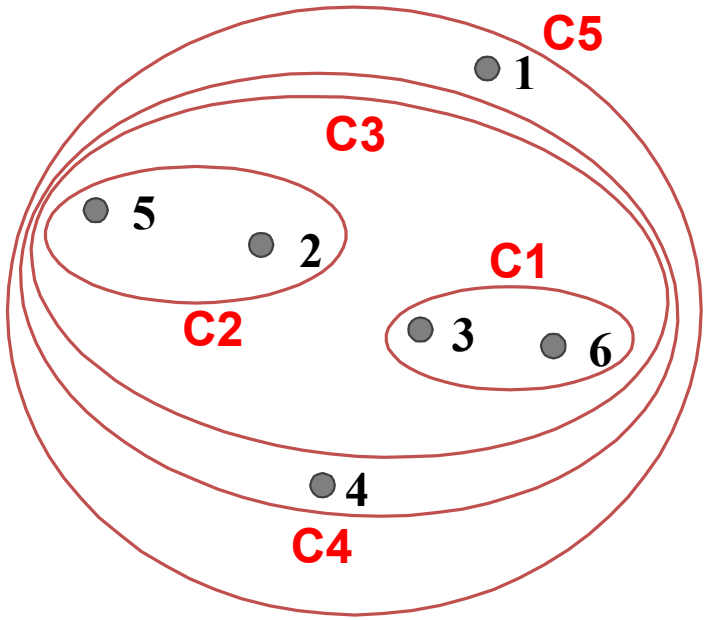
- Proximity of two clusters is based on the two closest points in the different clusters
  - Determined by one pair of points, i.e., by one link in the proximity graph



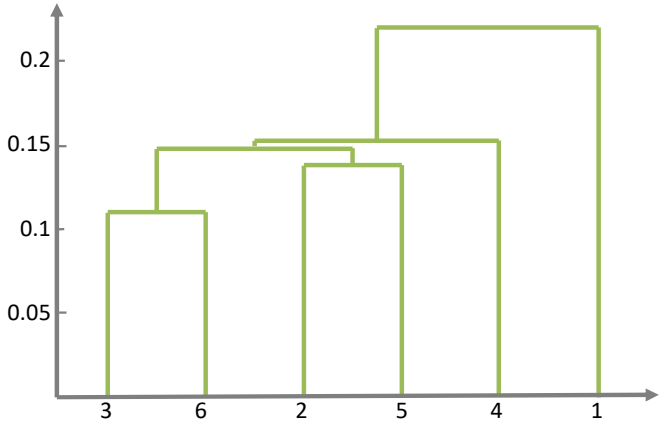
Distance Matrix:

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

Hierarchical Clustering: **MIN**



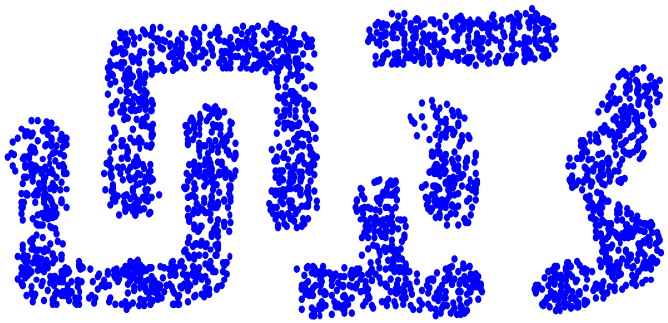
Nested Clusters



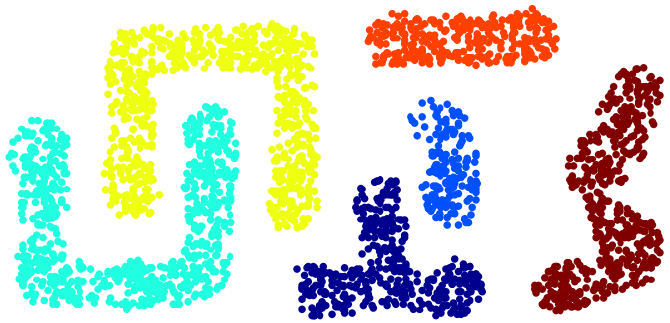
Dendrogram

Strength of MIN

- Can handle non-elliptical shapes



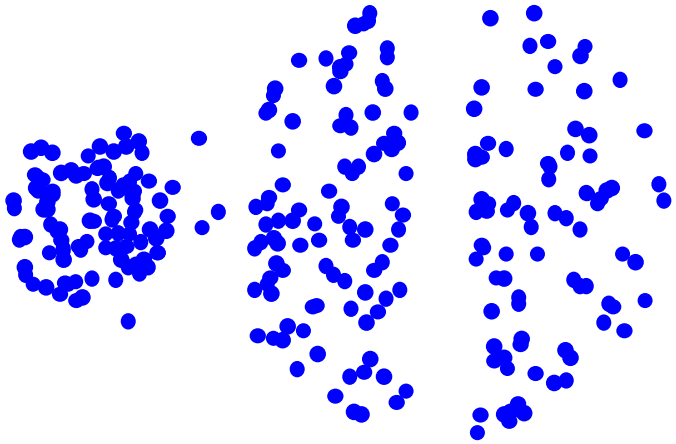
Original Points



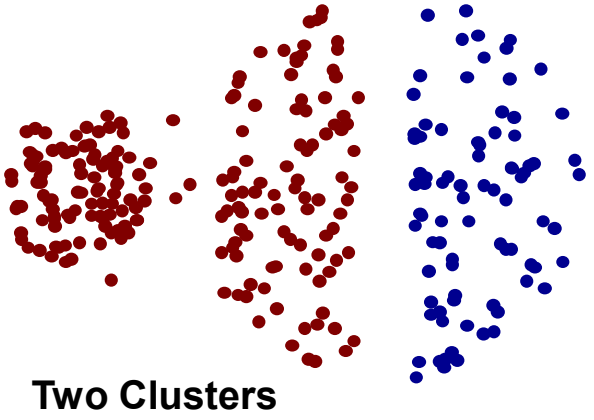
Six Clusters

Limitations of MIN

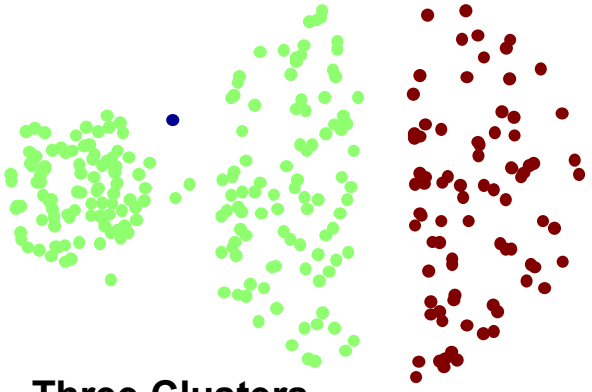
- Sensitive to noise and outliers



Original Points



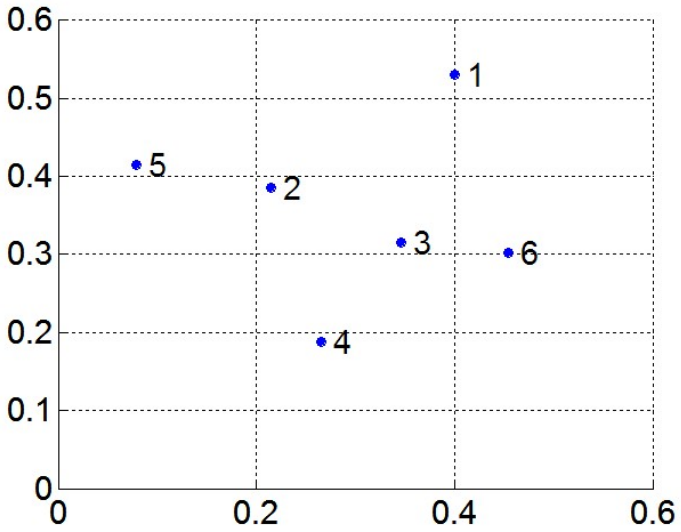
Two Clusters



Three Clusters

MAX or Complete Linkage

- Proximity of two clusters is based on the two most distant points in the different clusters
  - Determined by all pairs of points in the two clusters

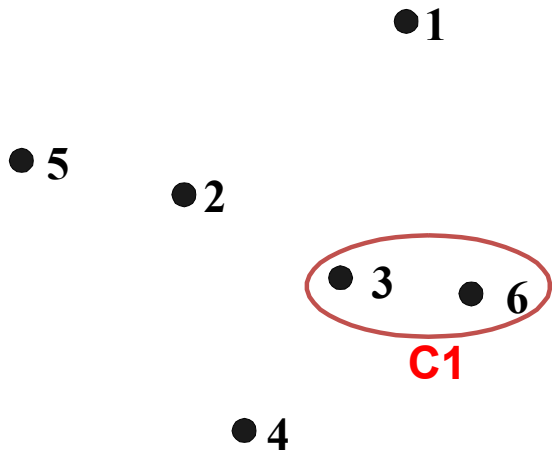


Distance Matrix:

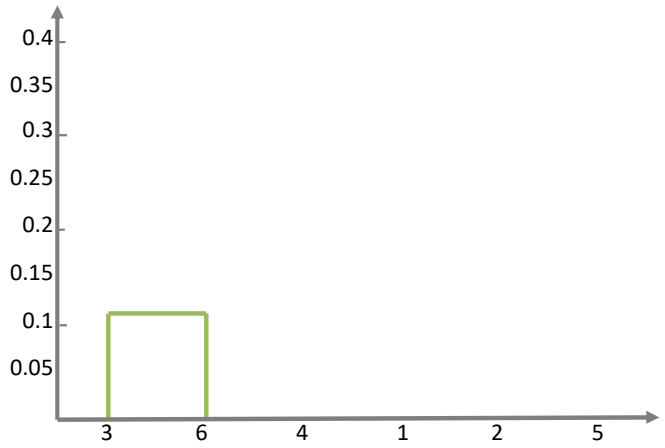
	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

Hierarchical Clustering: **MAX**

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00



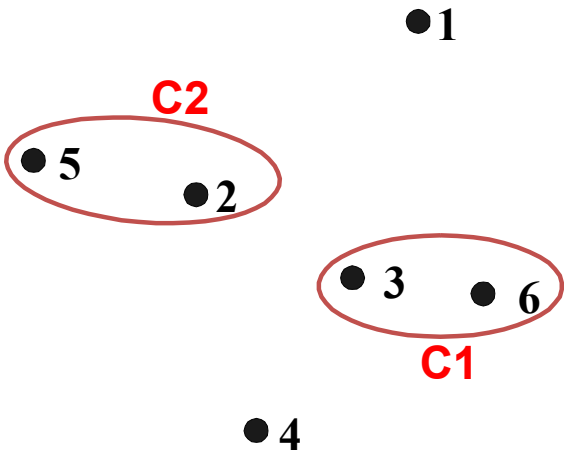
Nested Clusters



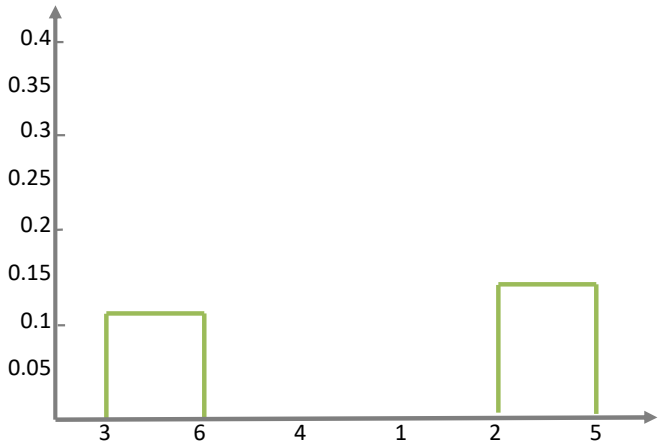
Dendrogram

Hierarchical Clustering: **MAX**

	p1	p2	C1	p4	p5
p1	0.00	0.24	0.23	0.37	0.34
p2	0.24	0.00	0.25	0.20	0.14
C1	0.23	0.25	0.00	0.22	0.39
p4	0.37	0.20	0.22	0.00	0.29
p5	0.34	0.14	0.39	0.29	0.00



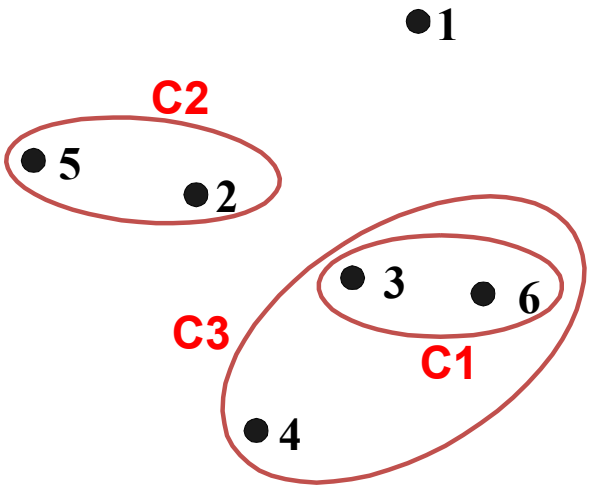
Nested Clusters



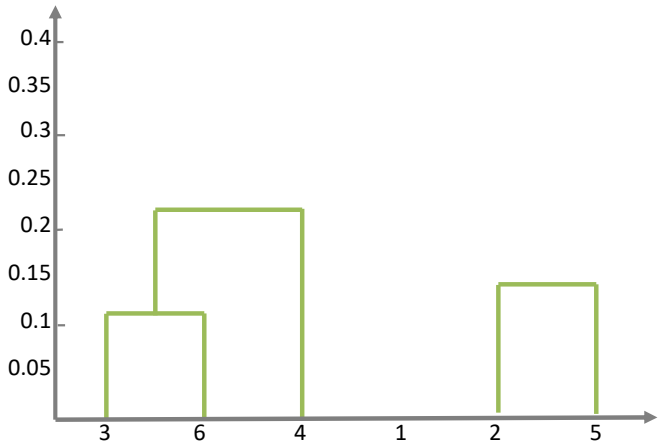
Dendrogram

Hierarchical Clustering: **MAX**

	p1	C2	C1	p4
p1	0.00	0.34	0.23	0.37
C2	0.34	0.00	0.39	0.29
C1	0.23	0.39	0.00	0.22
p4	0.37	0.29	0.22	0.00



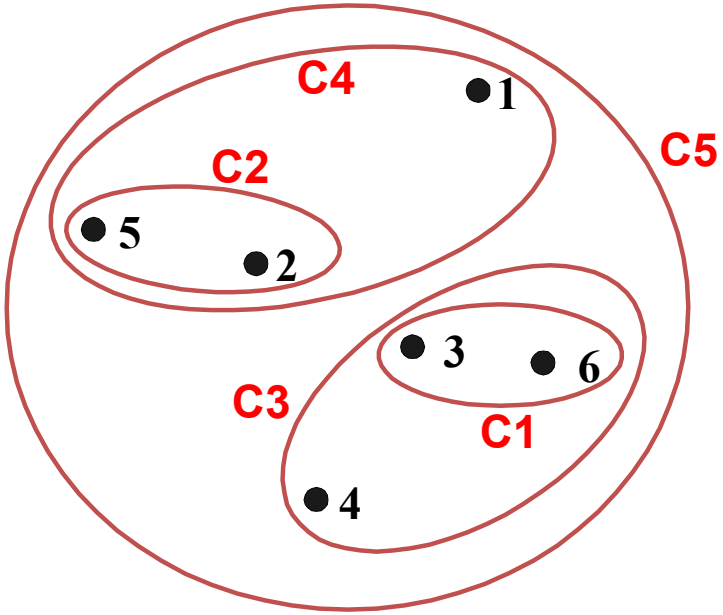
Nested Clusters



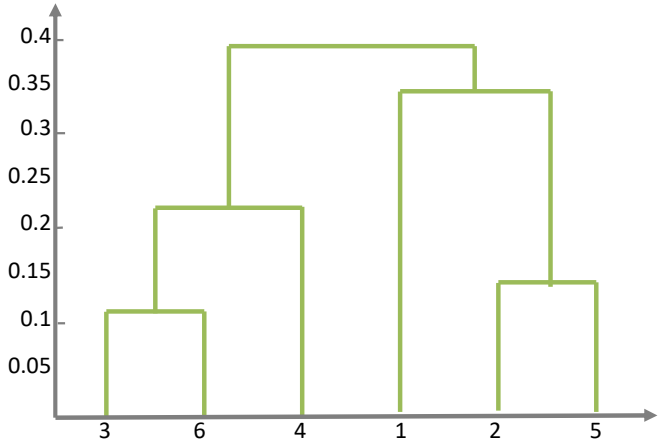
Dendrogram

Hierarchical Clustering: **MAX**

	p1	C2	C3
p1	0.00	0.34	0.37
C2	0.34	0.00	0.39
C3	0.37	0.39	0.00



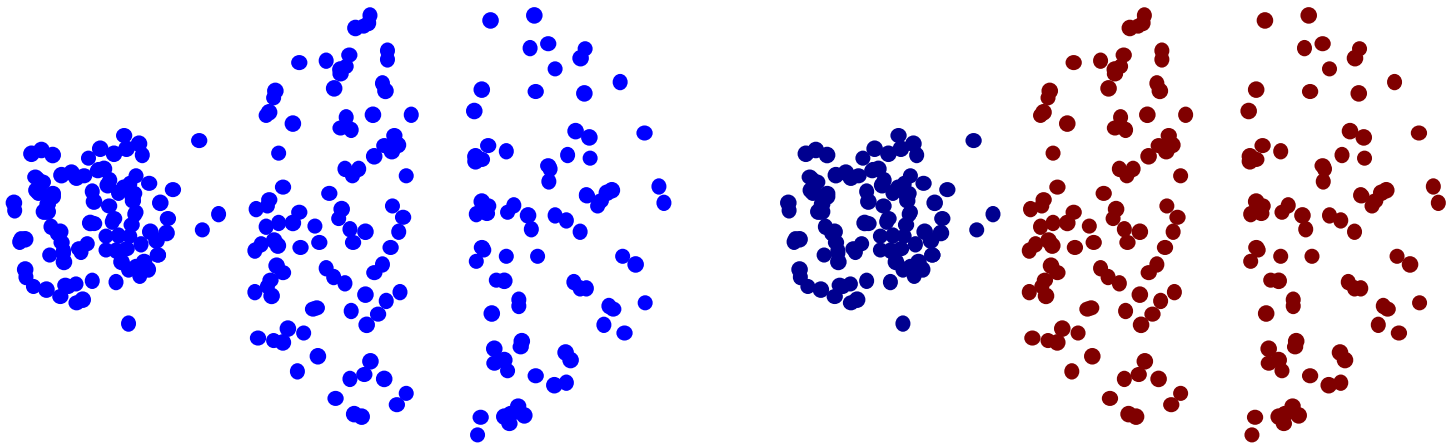
Nested Clusters



Dendrogram

Strength of MAX

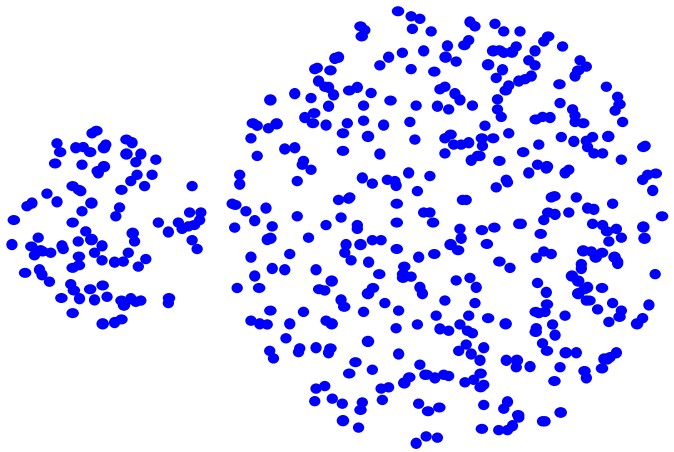
- Less susceptible to noise and outliers



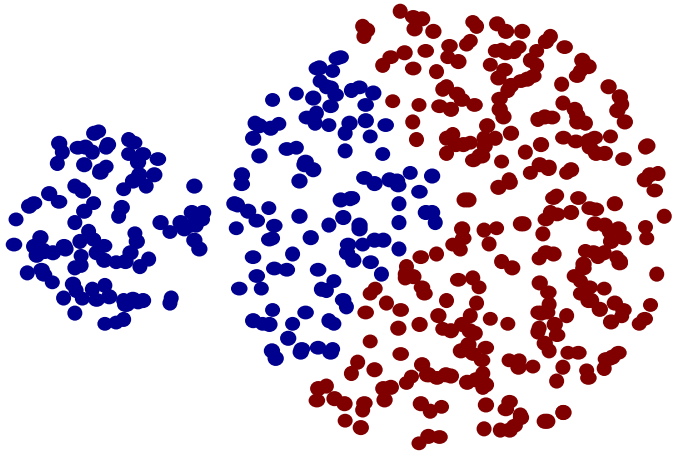
Original Points **Two Clusters**

Limitations of MAX

- Tends to break large clusters
- Biased towards globular clusters



Original Points



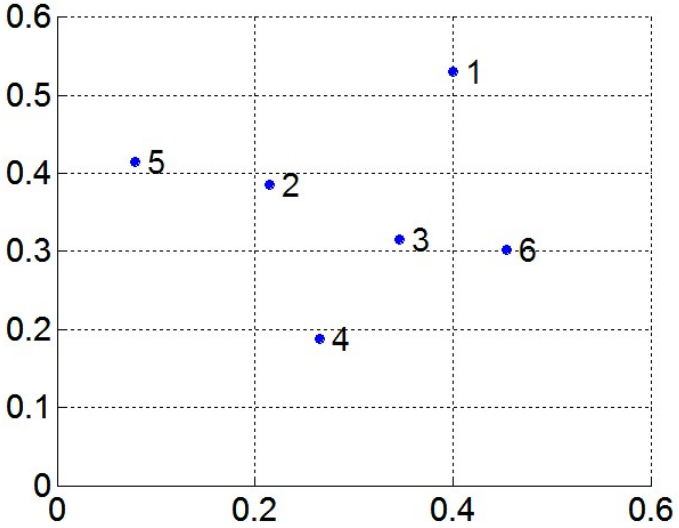
Two Clusters

Group Average

- Proximity of two clusters is the average of pairwise proximity between points in the two clusters.

$$\text{proximity}(\text{Cluster}_i, \text{Cluster}_j) = \frac{\sum_{\substack{p_i \in \text{Cluster}_i \\ p_j \in \text{Cluster}_j}} \text{proximity}(p_i, p_j)}{|\text{Cluster}_i| \times |\text{Cluster}_j|}$$

- Need to use average connectivity for scalability since total proximity favors large clusters

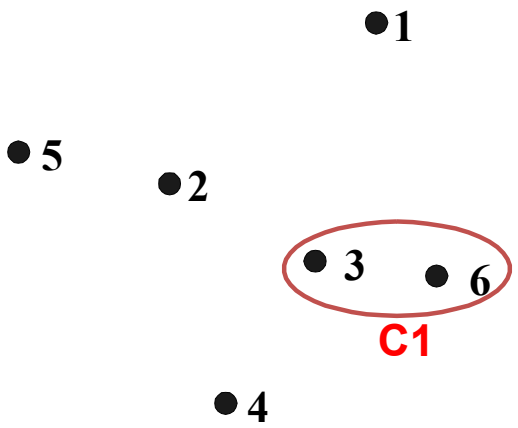


Distance Matrix:

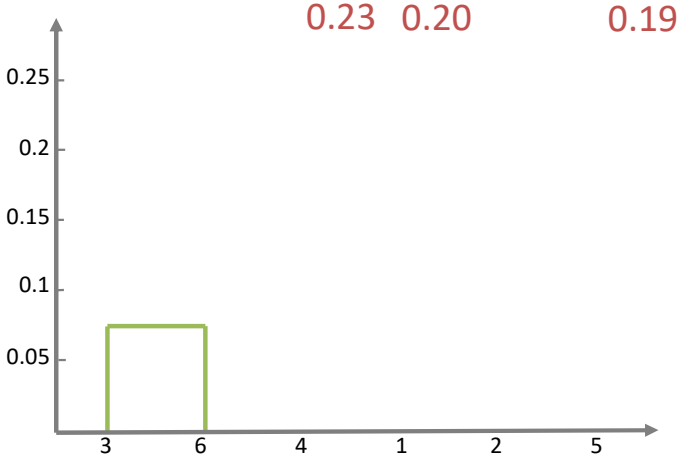
	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

Hierarchical Clustering: **Group Average**

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00



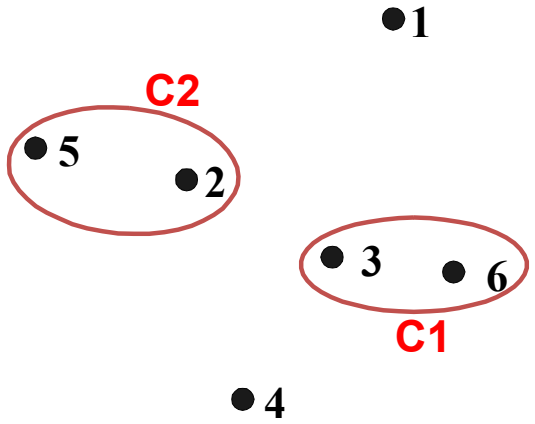
Nested Clusters



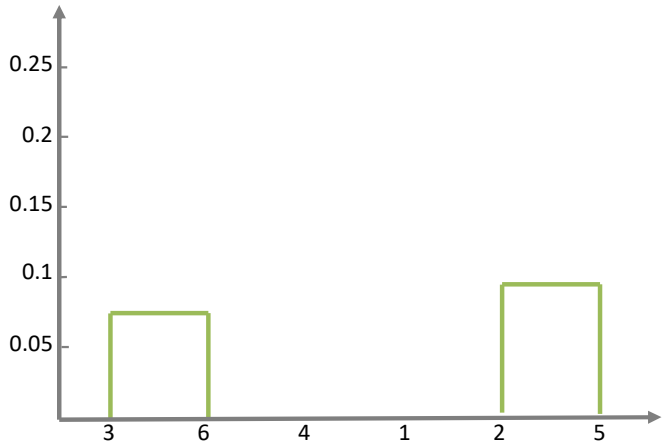
Dendrogram

Hierarchical Clustering: **Group Average**

	p1	p2	C1	p4	p5
p1	0.00				
p2	0.24	0.00			
C1	0.23	0.2	0.00		
p4	0.37	0.20	0.19	0.00	
p5	0.34	0.14	0.34	0.29	0.00



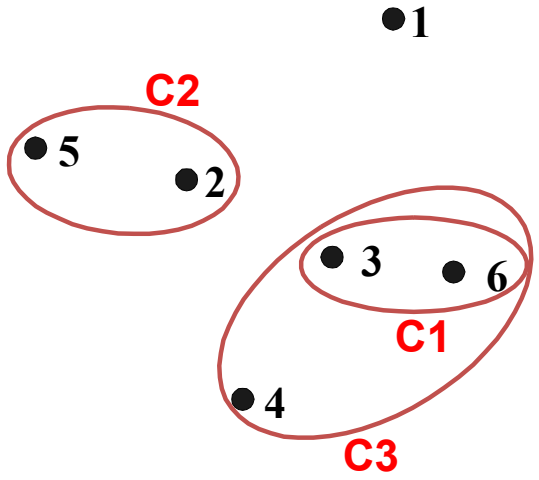
Nested Clusters



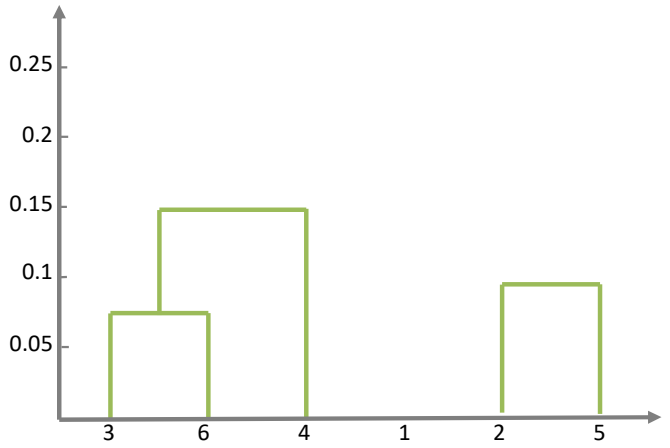
Dendrogram

Hierarchical Clustering: **Group Average**

	p1	C2	C1	p4
p1	0.00			
C2	0.29	0.00		
C1	0.23	0.27	0.00	
p4	0.37	0.25	0.19	0.00



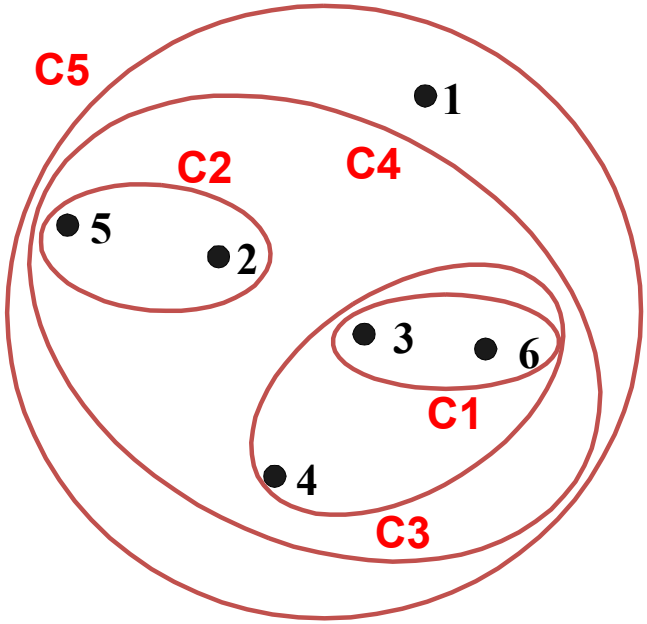
Nested Clusters



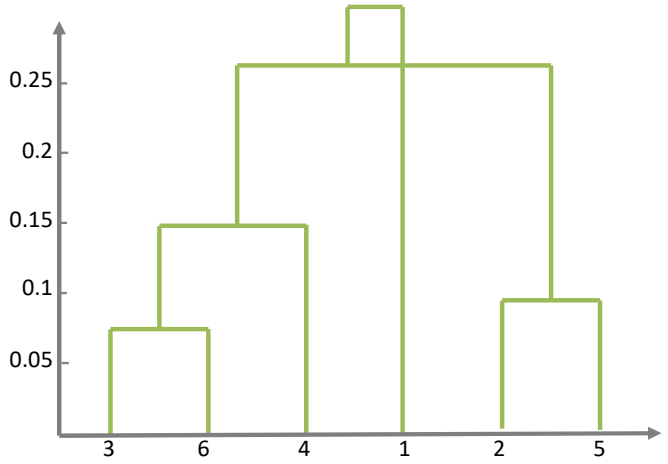
Dendrogram

Hierarchical Clustering: **Group Average**

	p1	C2	C3
p1	0.00		
C2	0.29	0.00	
C3	0.30	0.26	0.00



Nested Clusters



Dendrogram

Hierarchical Clustering: **Group Average**

- Compromise between Single and Complete Link

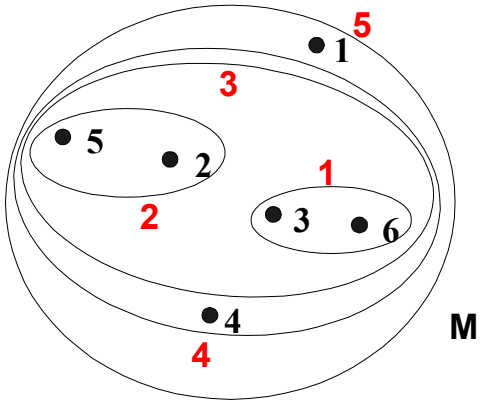
- Strengths
  - Less susceptible to noise and outliers

- Limitations
  - Biased towards globular clusters

Cluster Similarity: Ward's Method

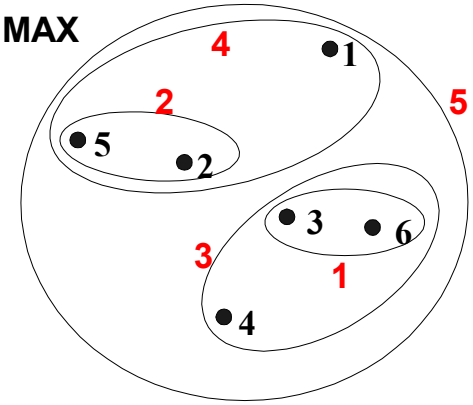
- Similarity of two clusters is based on the increase in squared error when two clusters are merged
  - Similar to group average if distance between points is distance squared
- Less susceptible to noise and outliers
- Biased towards globular clusters
- Hierarchical analogue of K-means
  - Can be used to initialize K-means

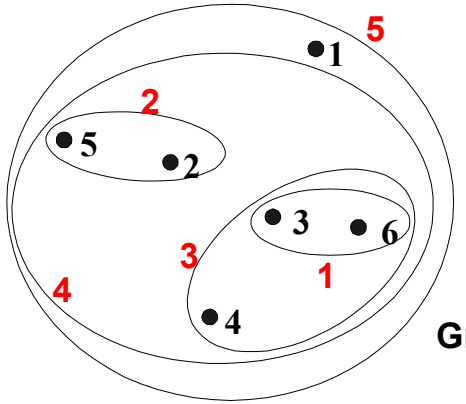
Hierarchical Clustering: **Comparison**



MIN

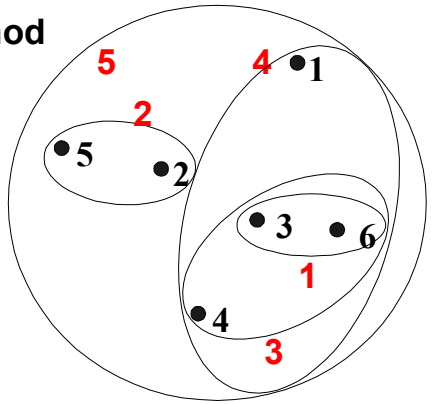
MAX





Group Average

Ward's Method



Hierarchical Clustering: **Problems and Limitations**

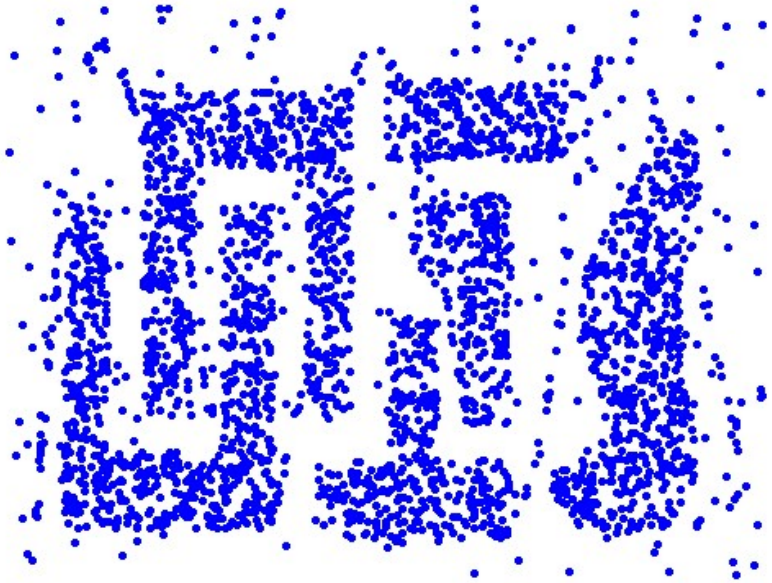
- Once a decision is made to combine two clusters, it cannot be undone

- No global objective function is directly minimized

- Different schemes have problems with one or more of the following:
  - Sensitivity to noise and outliers
  - Difficulty handling clusters of different sizes and non-globular shapes
  - Breaking large clusters

Density Based Clustering

- Clusters are regions of high density that are separated from one another by regions on low density.

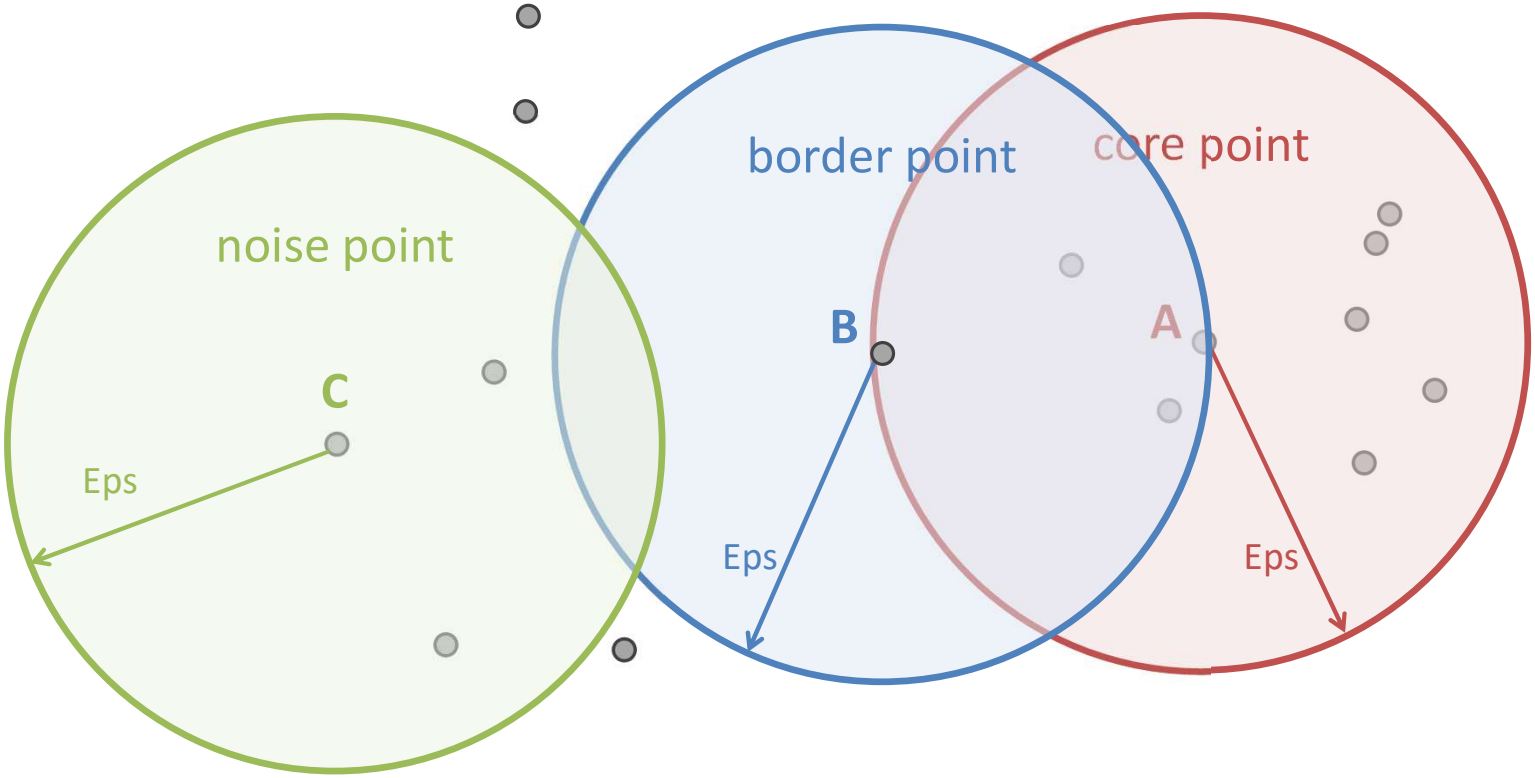


DBSCAN

- DBSCAN is a density-based algorithm.
  - **Density** = number of points within a specified radius (Eps)
  - A point is a **core point** if it has at least a specified number of points (MinPts) within Eps
    - These are points that are at the interior of a cluster
    - Counts the point itself
  - A **border point** is not a core point, but is in the neighborhood of a core point
  - A **noise point** is any point that is not a core point or a border point

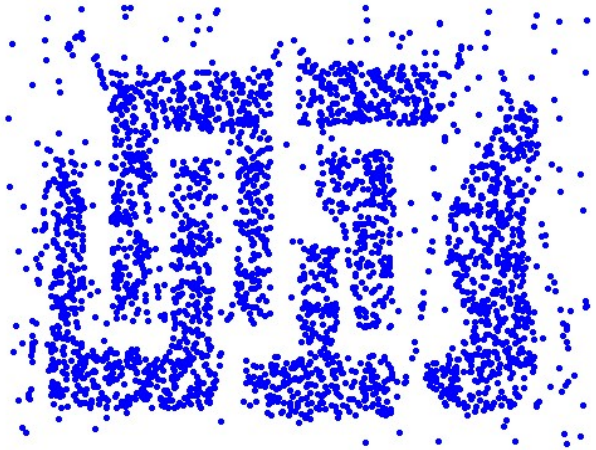
DBSCAN: Core, Border, and Noise Points

MinPts = 7

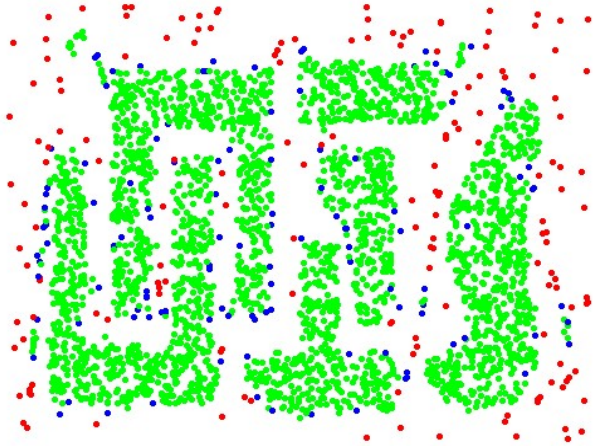


DBSCAN: Core, Border and Noise Points

- Eps = 10, MinPts = 4



Original Points



Point types: core, border, noise

DBSCAN Algorithm

- Form clusters using core points, and assign border points to one of its neighboring clusters

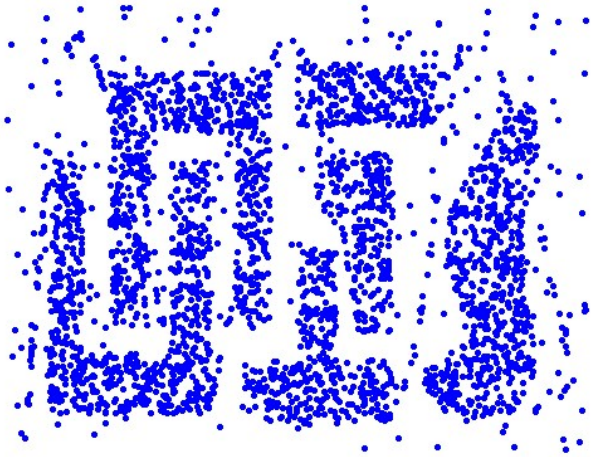
---

1. Label all points as core, border, or noise points.
2. Eliminate noise points.
3. Put an edge between all core points within a distance *Eps* of each other.
4. Make each group of connected core points into a separate cluster.
5. Assign each border point to one of the clusters of its associated core points

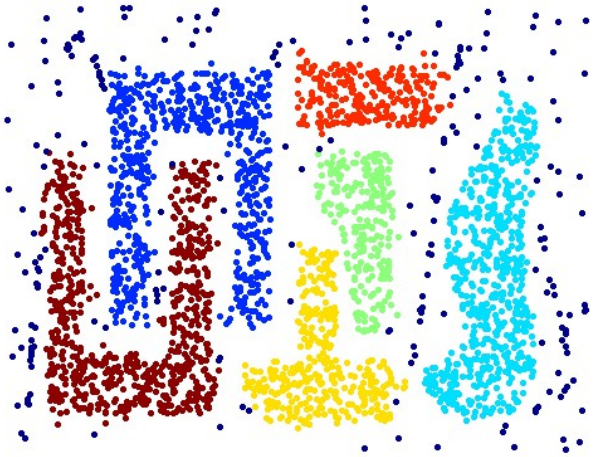
---

When DBSCAN Works Well

- Resistant to Noise
- Can handle clusters of different shapes and sizes



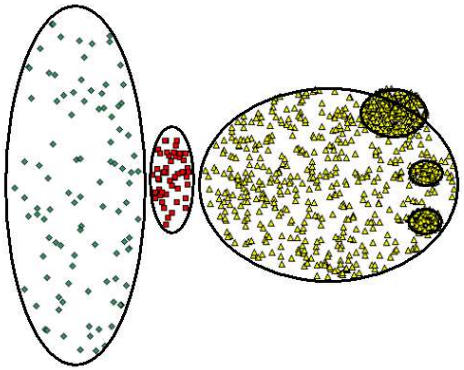
Original Points



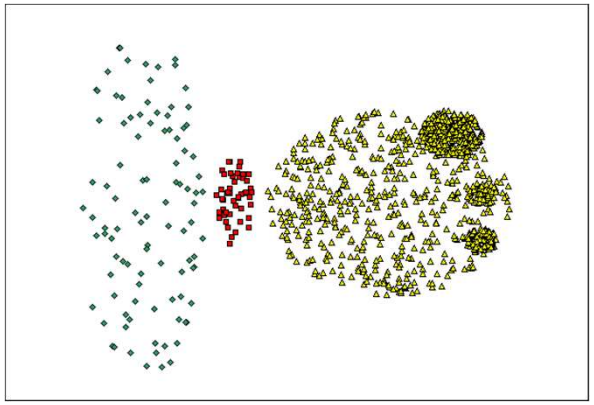
Clusters

When DBSCAN Does NOT Work Well

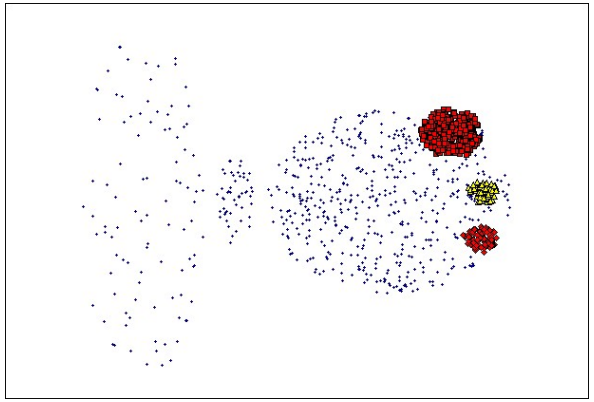
- Varying densities
- High-dimensional data



Original Points



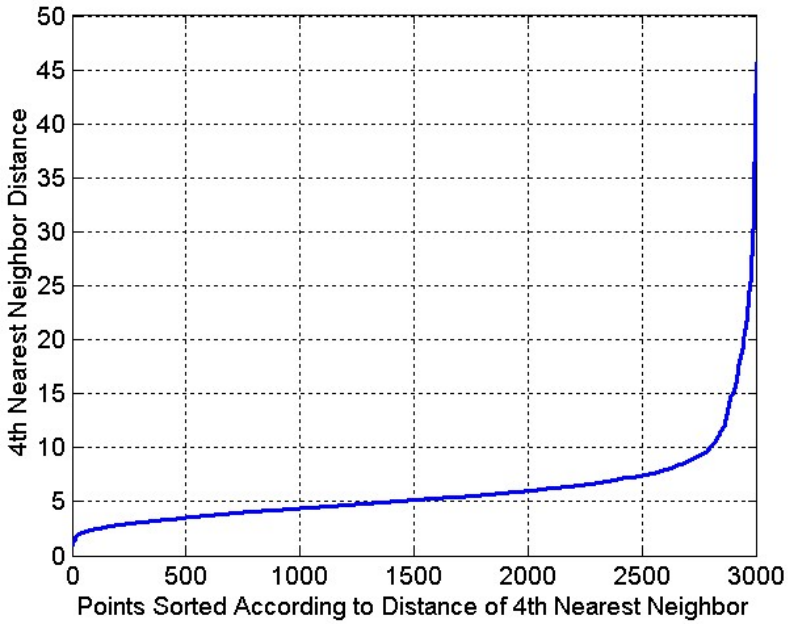
(MinPts = 4, Eps = 9.92)



(MinPts = 4, Eps = 9.75)

DBSCAN: Determining EPS and MinPts

- Idea is that for points in a cluster, their k^{th} nearest neighbors are at close distance
- Noise points have the kth nearest neighbor at farther distance
- So, plot sorted distance of every point to its kth nearest neighbor

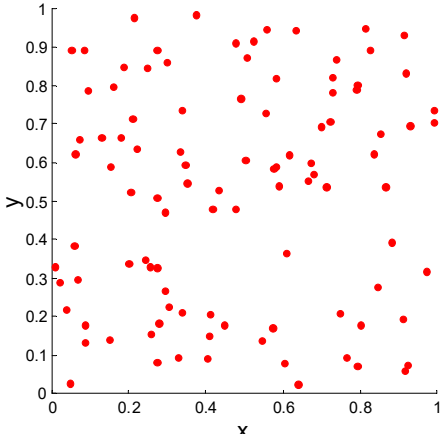


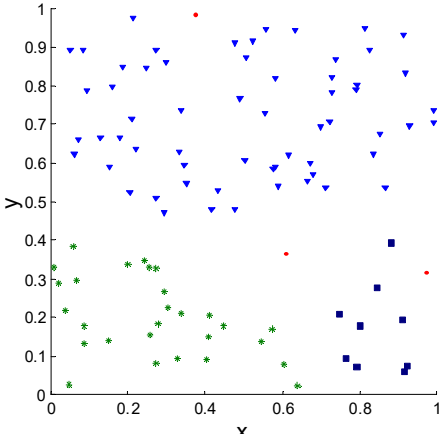
Cluster Validity

- For supervised classification we have a variety of measures to evaluate how good our model is
  - Accuracy, precision, recall
- For cluster analysis, the analogous question is how to evaluate the "goodness" of the resulting clusters?
- But "clusters are in the eye of the beholder"!
  - In practice the clusters we find are defined by the clustering algorithm
- Then why do we want to evaluate them?
  - To avoid finding patterns in noise
  - To compare clustering algorithms
  - To compare two sets of clusters
  - To compare two clusters

Clusters found in Random Data

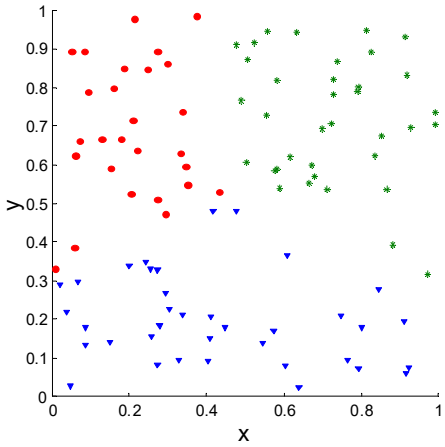
Random Points

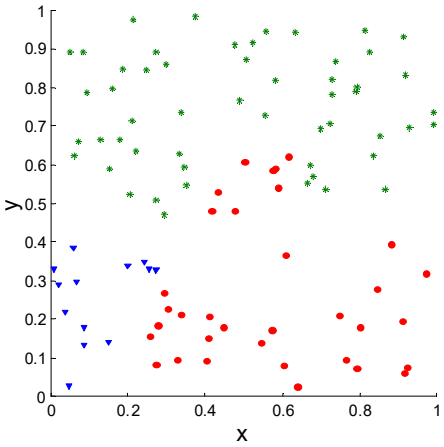




DBSCAN

K-means

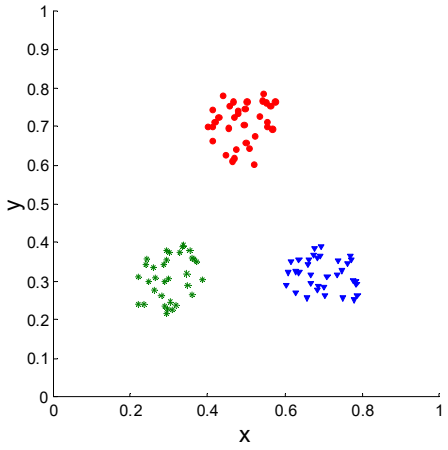




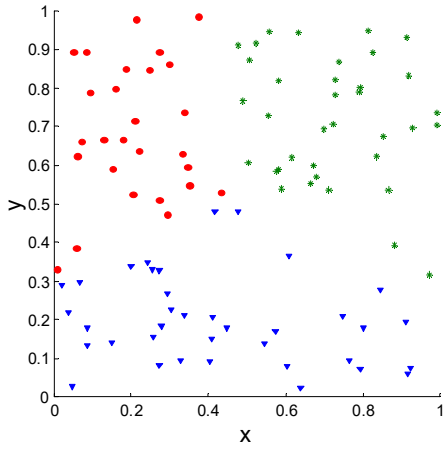
Complete Link

Measuring Cluster Validity Via Correlation

- Correlation of ideal similarity and proximity matrices for the K-means clusterings of the following two data sets.



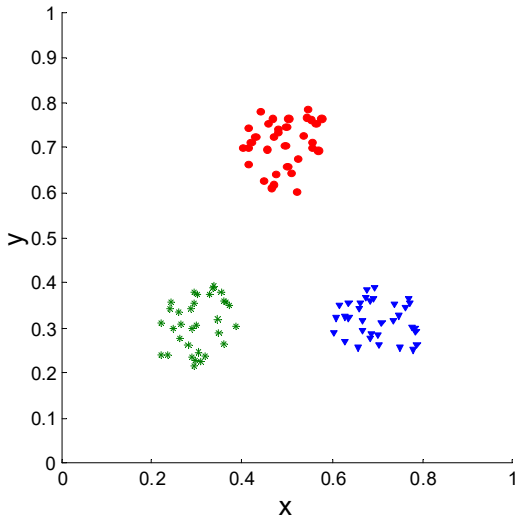
Corr = −0.9235

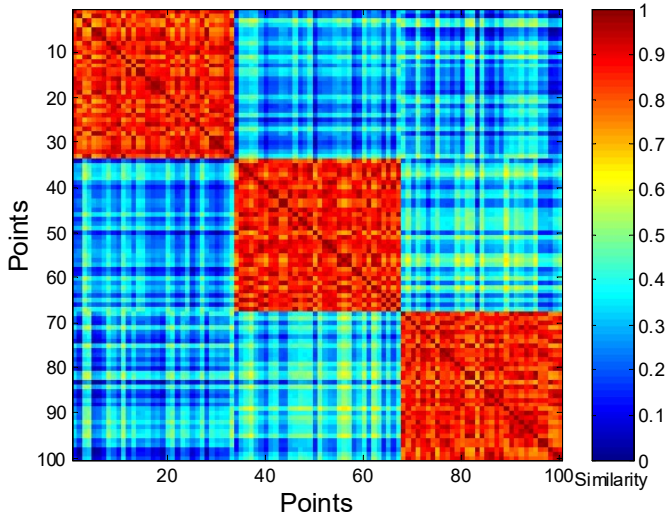


Corr = −0.5810

Using Similarity Matrix for Cluster Validation

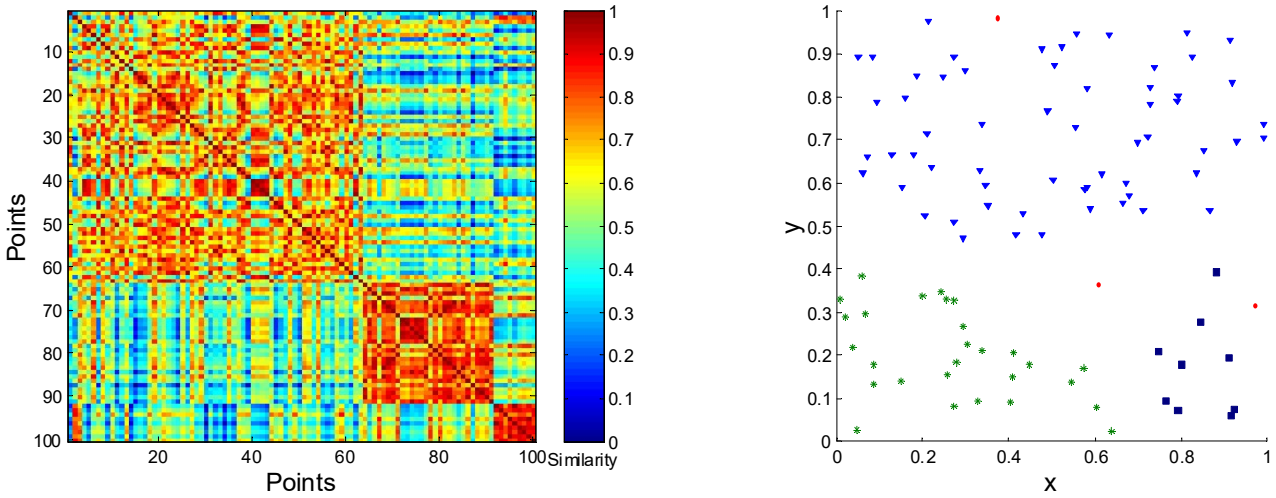
- Order the similarity matrix with respect to cluster labels and inspect visually.





Using Similarity Matrix for Cluster Validation

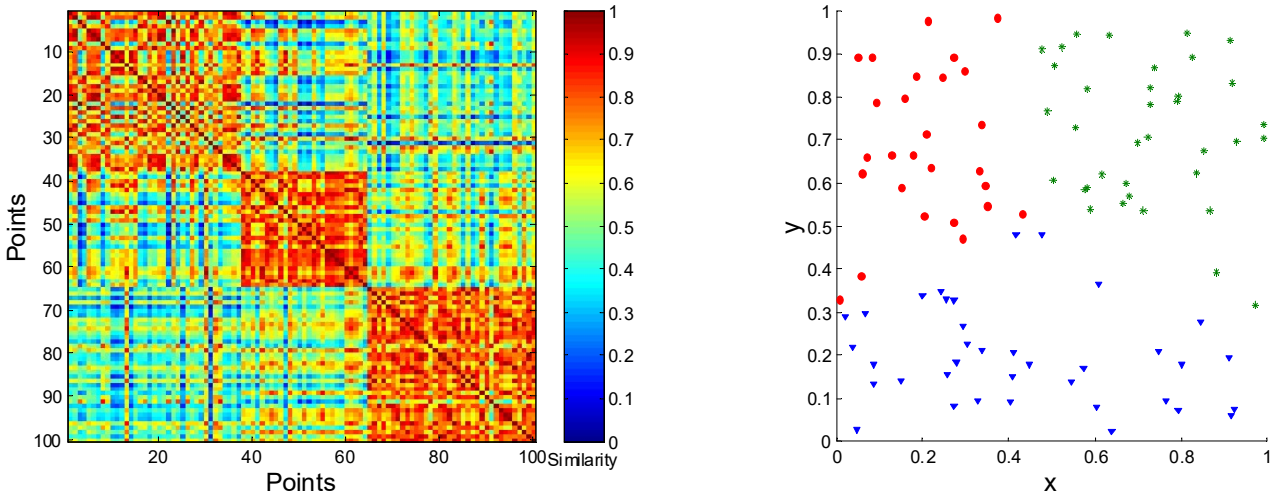
- Clusters in random data are not so crisp



DBSCAN

Using Similarity Matrix for Cluster Validation

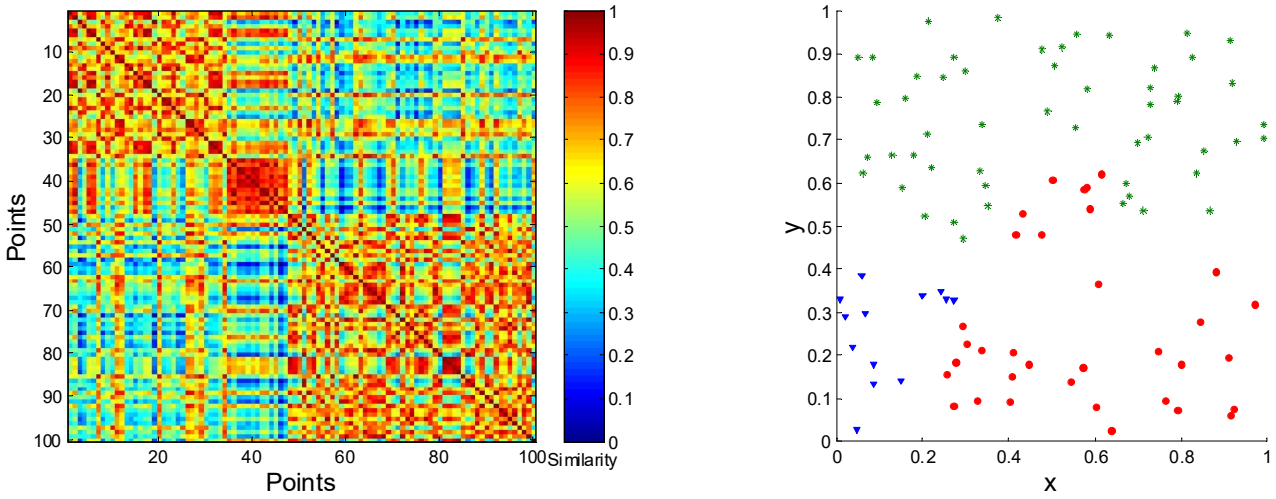
- Clusters in random data are not so crisp



K-means

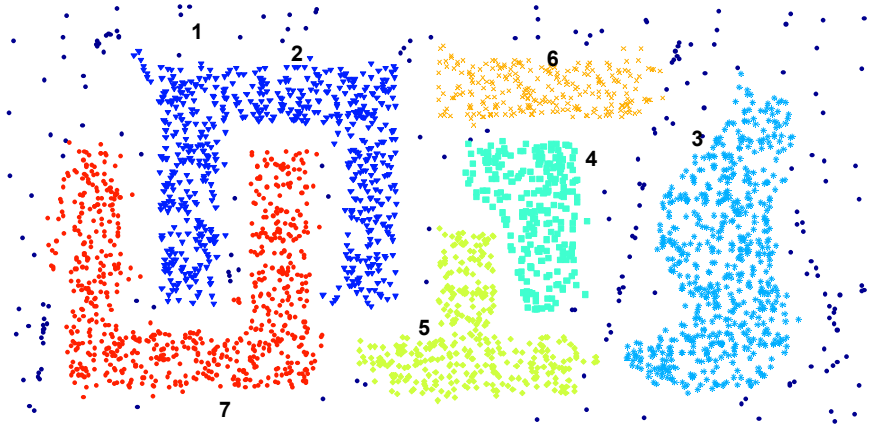
Using Similarity Matrix for Cluster Validation

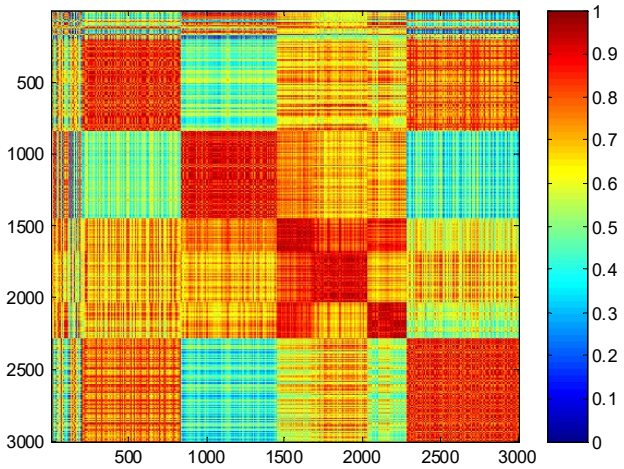
- Clusters in random data are not so crisp



Complete Link

Using Similarity Matrix for Cluster Validation

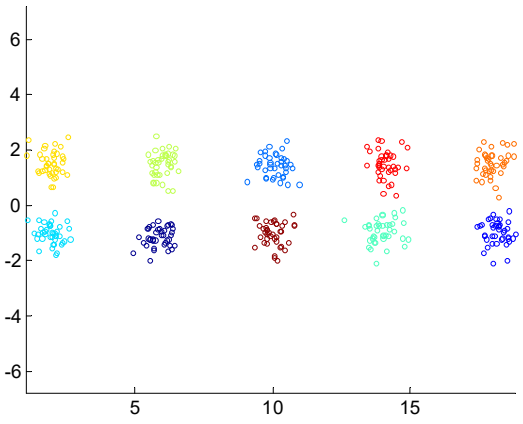


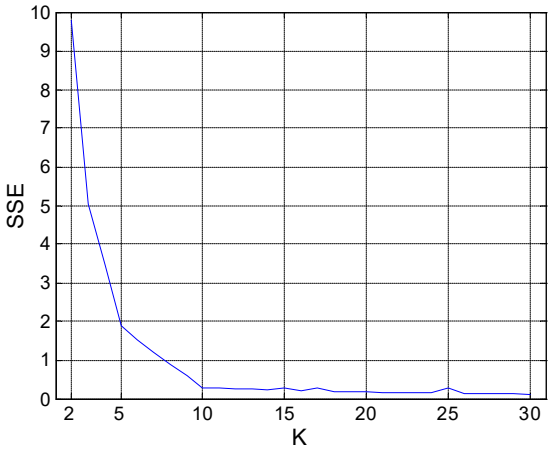


DBSCAN

Internal Measures: SSE

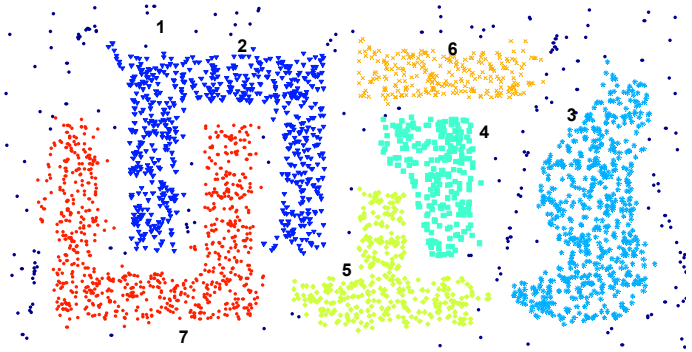
- Clusters in more complicated figures aren't well separated
- **Internal Index**: Used to measure the goodness of a clustering structure without respect to external information
  - SSE
  - SSE is good for comparing two clusterings or two clusters
  - Can also be used to estimate the number of clusters

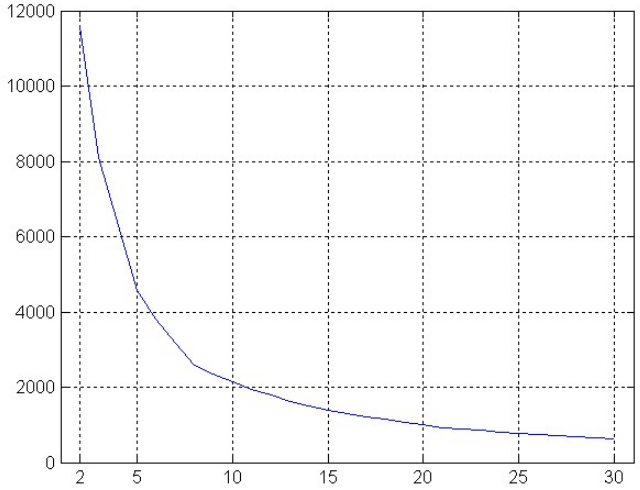




Internal Measures: SSE

- SSE curve for a more complicated data set





SSE of clusters found using K-means

Autoencoders

REVIEW: **Neural Networks as function approximation**

- Given an input x and an output y there exists a mapping from input space to output space as follows:

$$x \rightarrow y$$
$$y = f(x) + \epsilon$$

- Our goal is to find an estimate of $f(x)$ which we will call $\hat{f}(x)$.

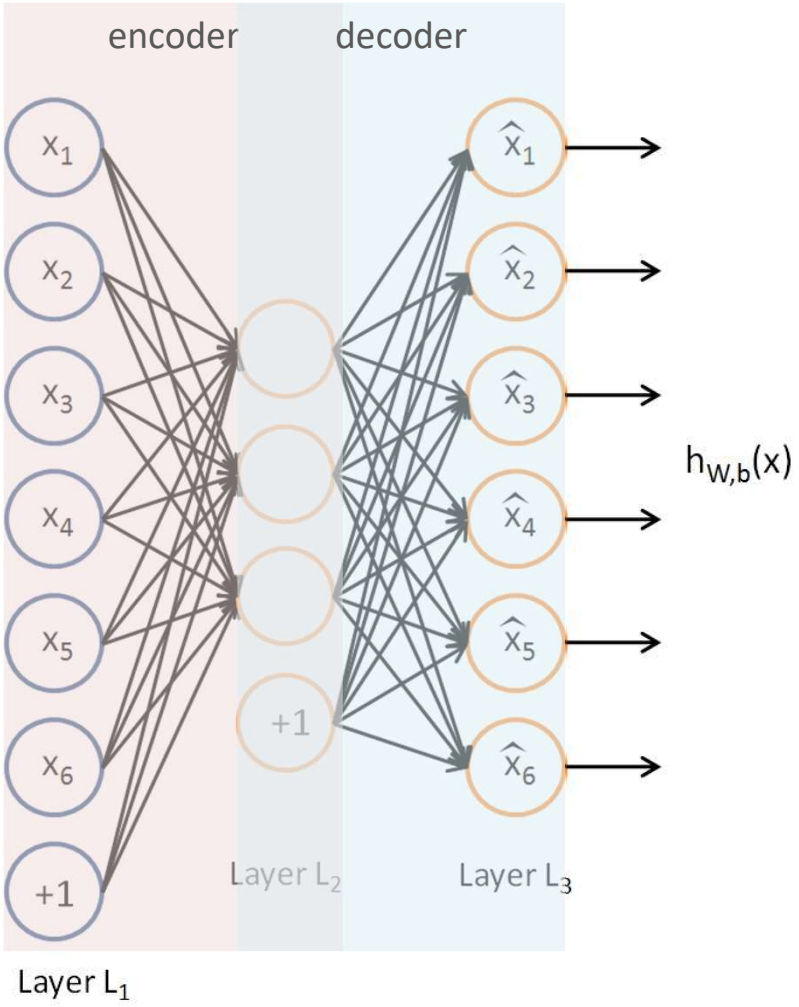
- Statistical learning or modeling is the process of finding $\hat{f}(x)$.
- Neural networks are one of many possible methods we can use to obtain the estimate $\hat{f}(x)$.

Autoencoders

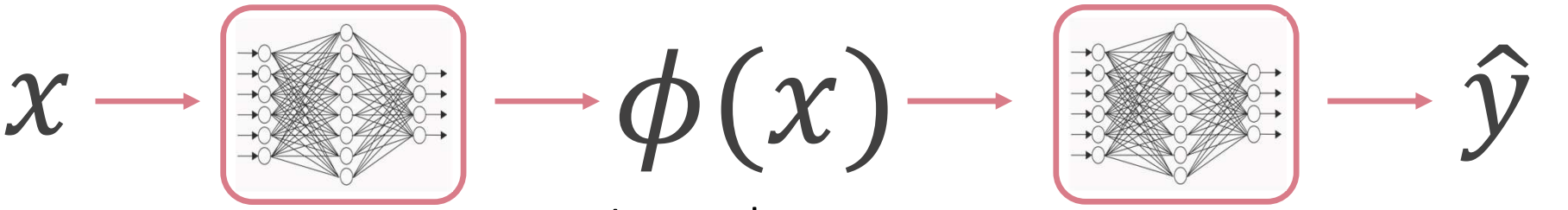
- An Autoencoder is a feedforward neural network that learns to predict the input itself in the output.

$$y^{(i)} = x^{(i)}$$

  - The input-to-hidden part corresponds to an encoder
  - The hidden-to-output part corresponds to a decoder.



Representational Learning



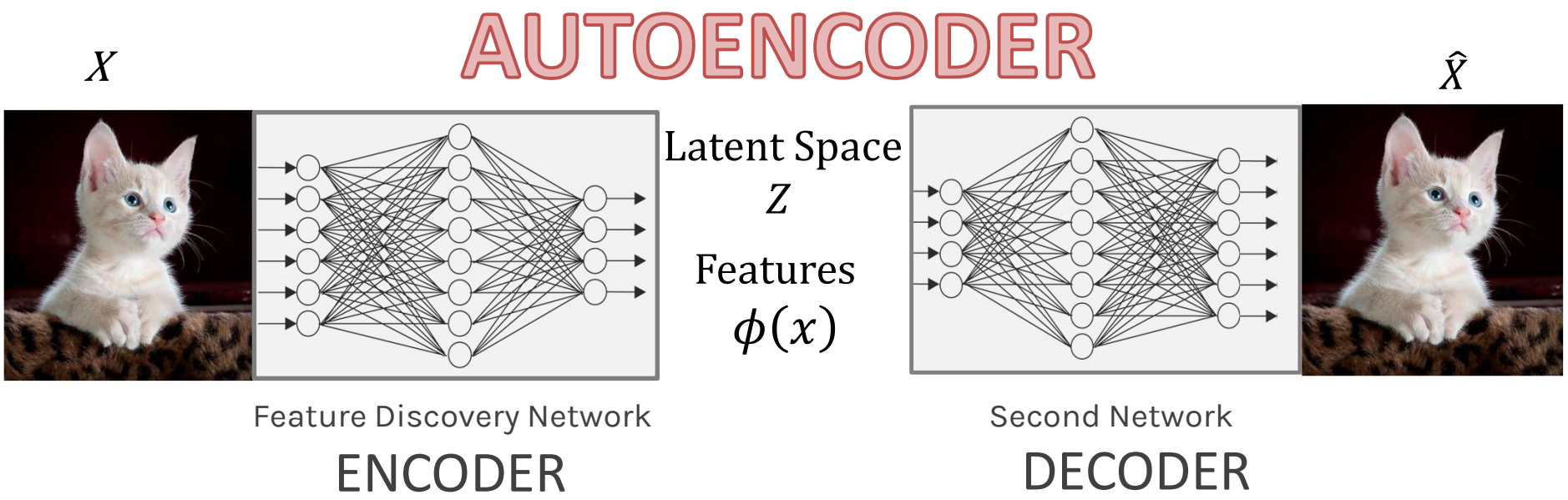
Learned representational features

Called Latent features
Latent space

Autoencoders learn the function:
$\hat{y} = x$

Representational Learning: **Self-supervised Learning**

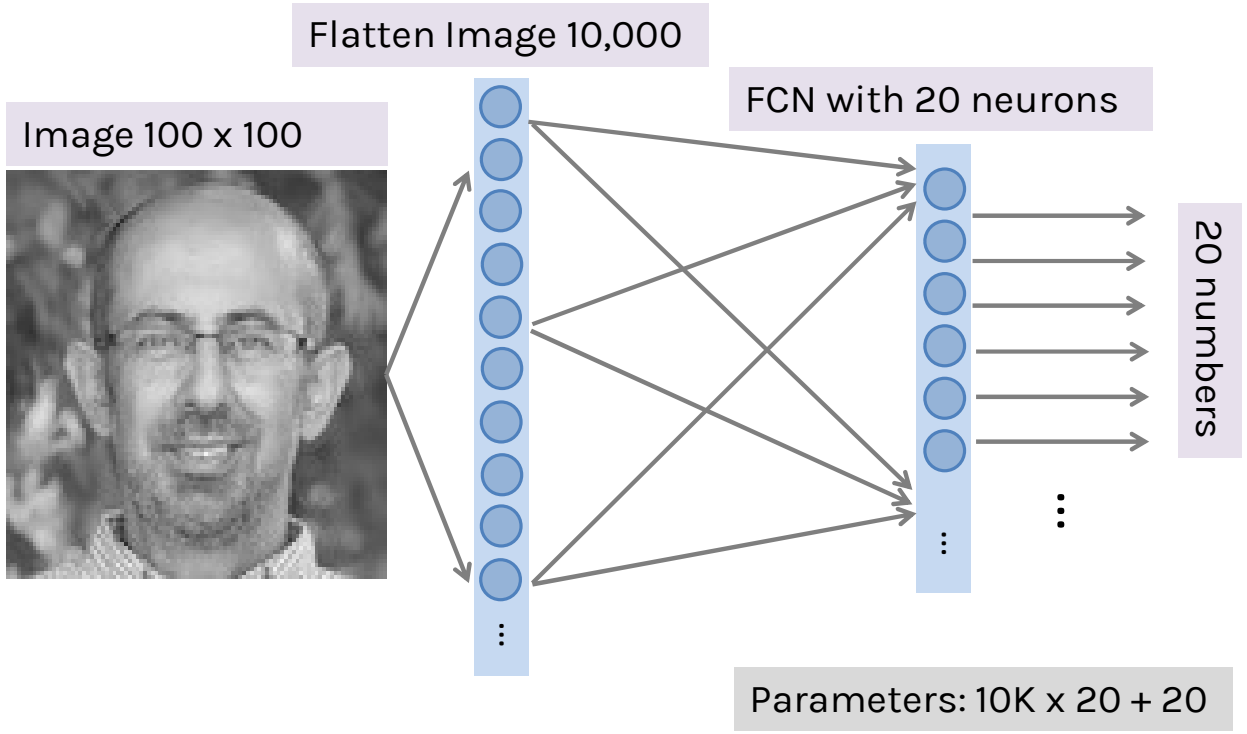
- We train the two networks by minimizing the **reconstruction** loss function: $\mathcal{L} = \sum(x_i - \hat{x}_i)^2$



The autoencoder gets that name because it automatically finds the best way to encode the input so that the decoded version is as close as possible to the input.

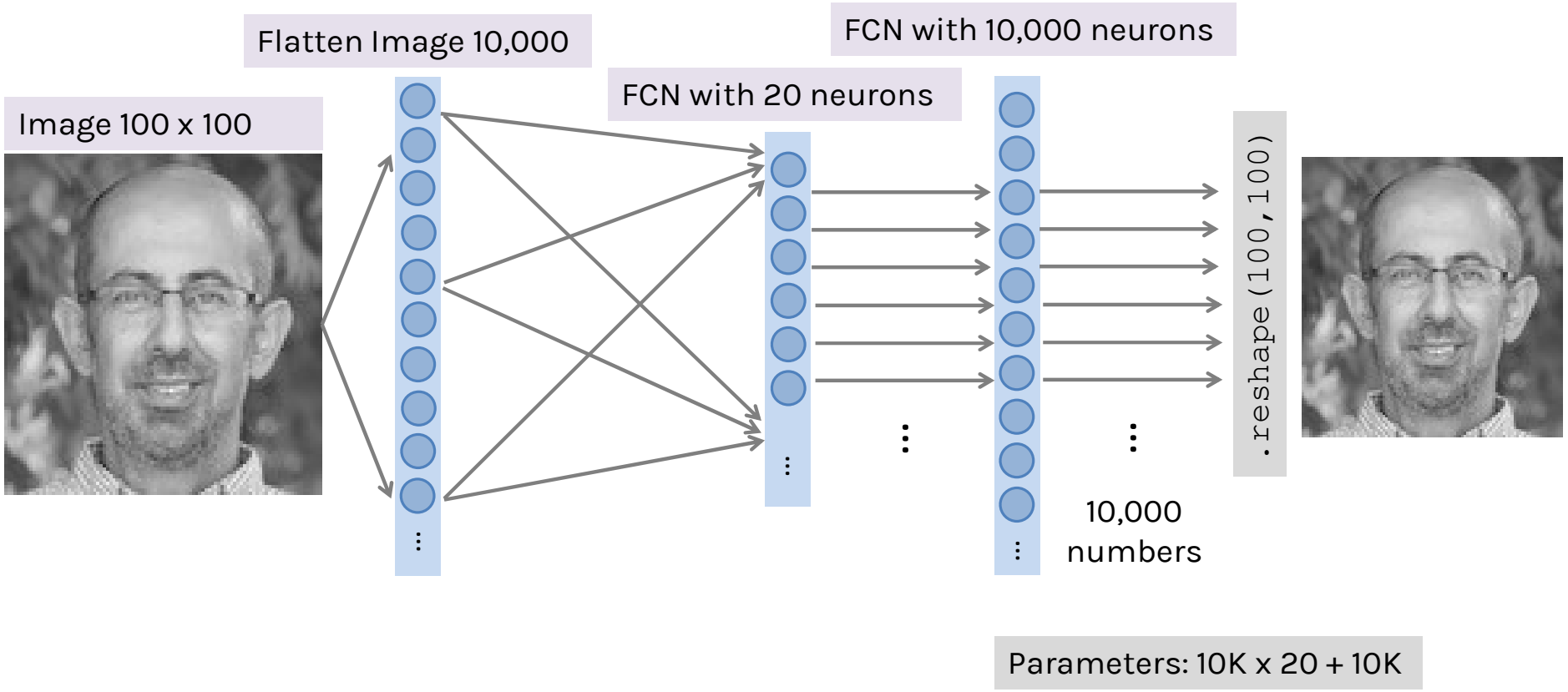
The simplest autoencoder

- **Encode** with a simple fully connected network (FCN)



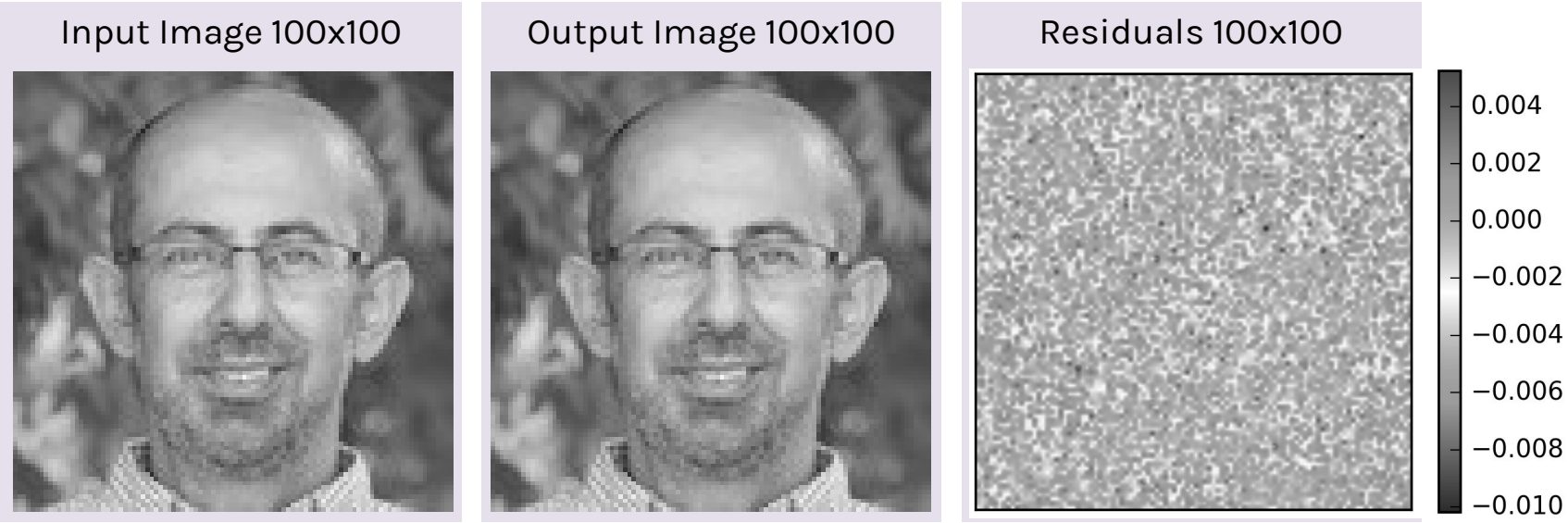
The simplest autoencoder

- Encode and decode together after training



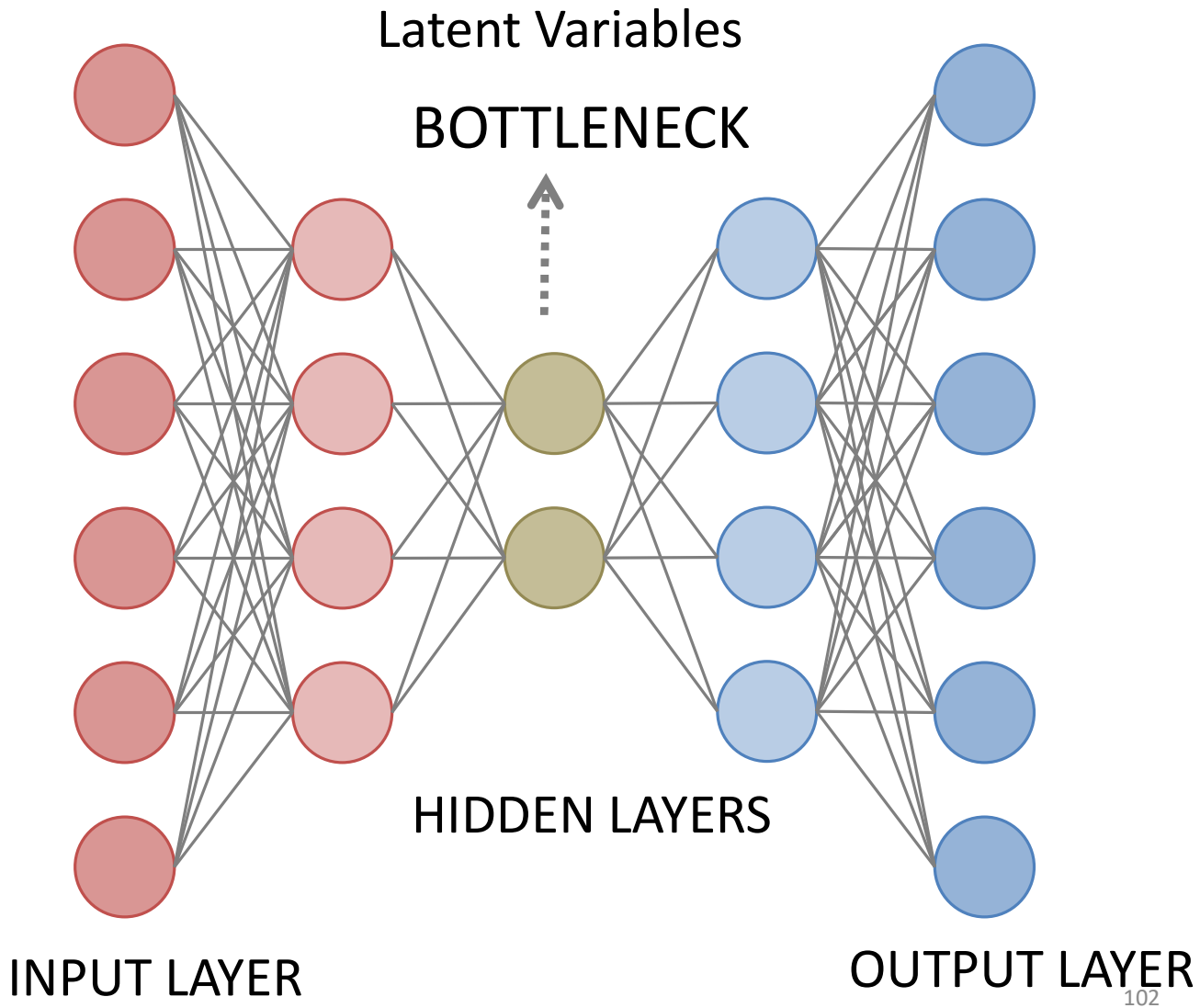
Autoencoders in action

- Comparing the input and output pixel by pixel.



Bottleneck

- We start with 10,000 elements
- We have 20 in the middle
- And 10,000 elements again at the end

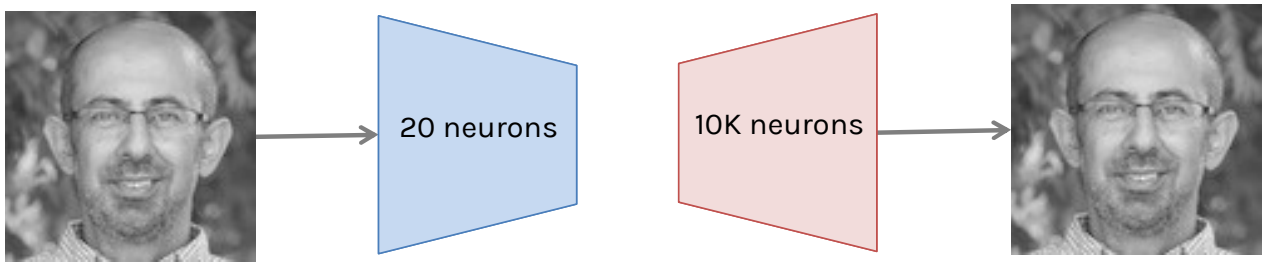


Unsupervised Learning of Latent Variables

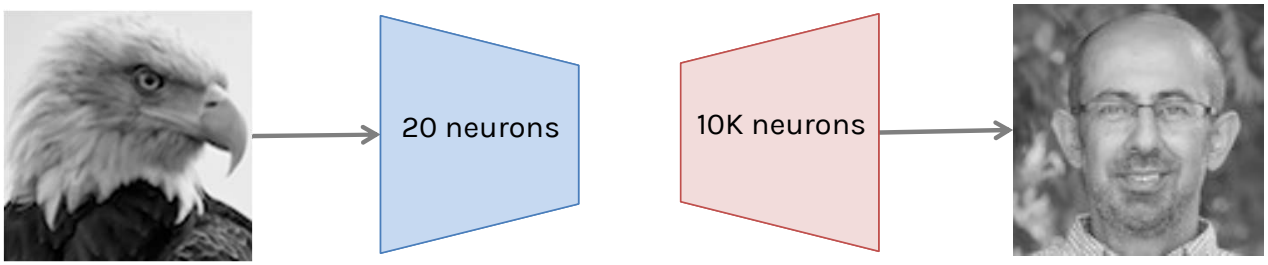
- It **sort-of is supervised** learning because we give the system explicit goal data (the **output** should be **the same** as the **input**), and it **sort-of isn't supervised** learning because we don't have any manually determined labels or targets on the inputs.

Autoencoders in action (cont)

- Passing a face to the trained autoencoder returns:



- How about if we input a photo of an eagle?



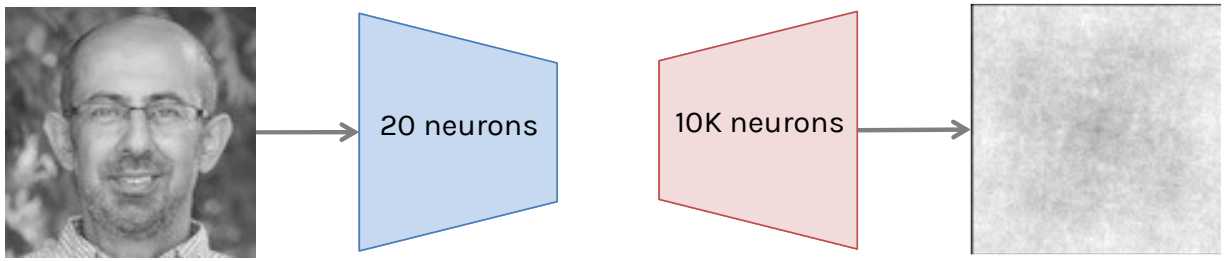
Autoencoders in action (cont)

- We must **train** with a variety of images.



Autoencoders in action (cont)

- After training with those images, let's test how well it generalizes:

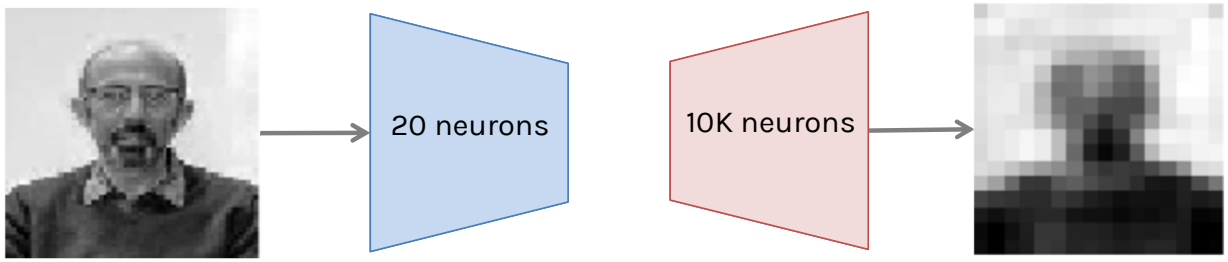


Network have never seen anything like this, so it is no surprise that could not reconstruct this.

Autoencoders in action (cont)

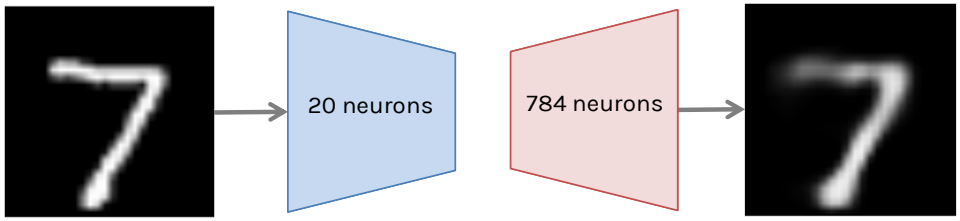
- We can use a better training set such as the Olivetti faces

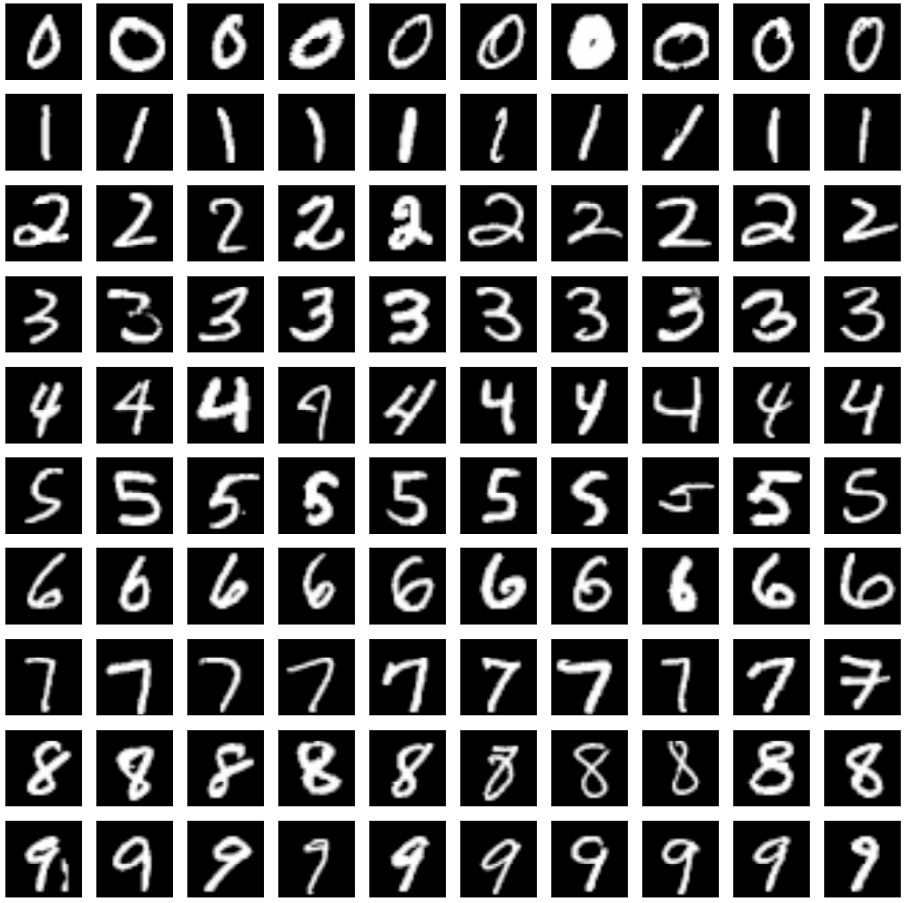




A better autoencoder

- **MNIST data**: train a simple AE with **one-layer FCN encoder** and **one-layer FCN decoder**





WHY DOES IT WORK?

20 latent variables

original

reconstructed



10 latent variables

original

reconstructed



2 latent variables

original

reconstructed

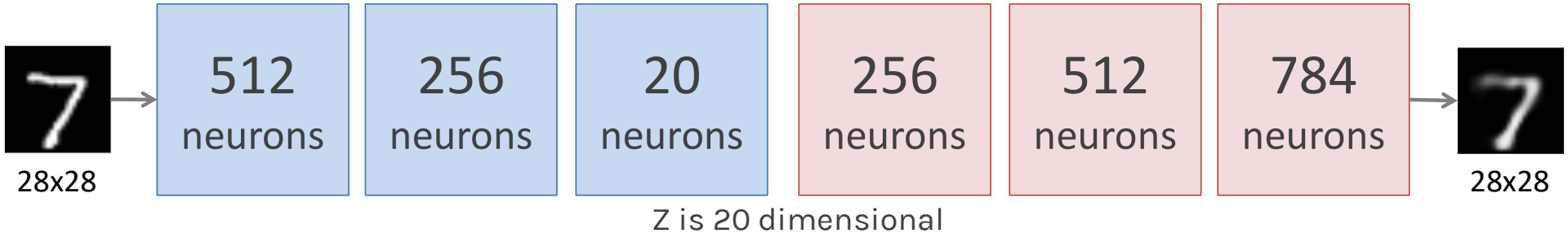


Image taken from A. Glassner, Deep Learning, Vol. 2: From Basics to Practice

Deeper

DEEPER IS BETTER

- For a better representation we can add neurons in one layer or go deeper.



Original Images



Reconstructed Images



Shallow 20 latent variables



Deeper 20 latent variables

Exploring autoencoders

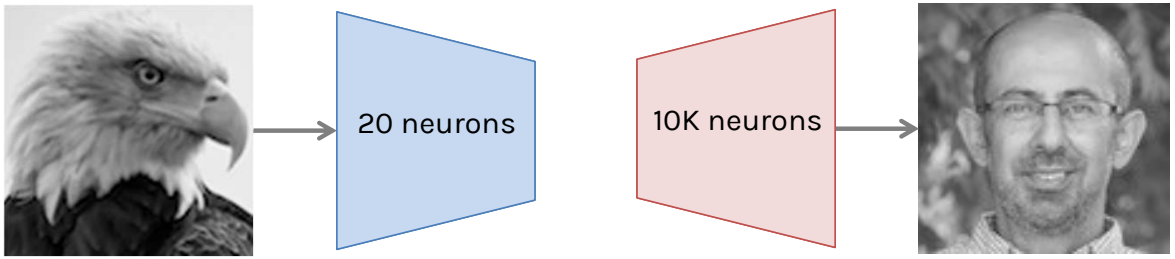
- How can be sure that there is information in the latent space?

2 latent variables

original

reconstructed





Exploring autoencoders

- For that let's explore the actual latent space.

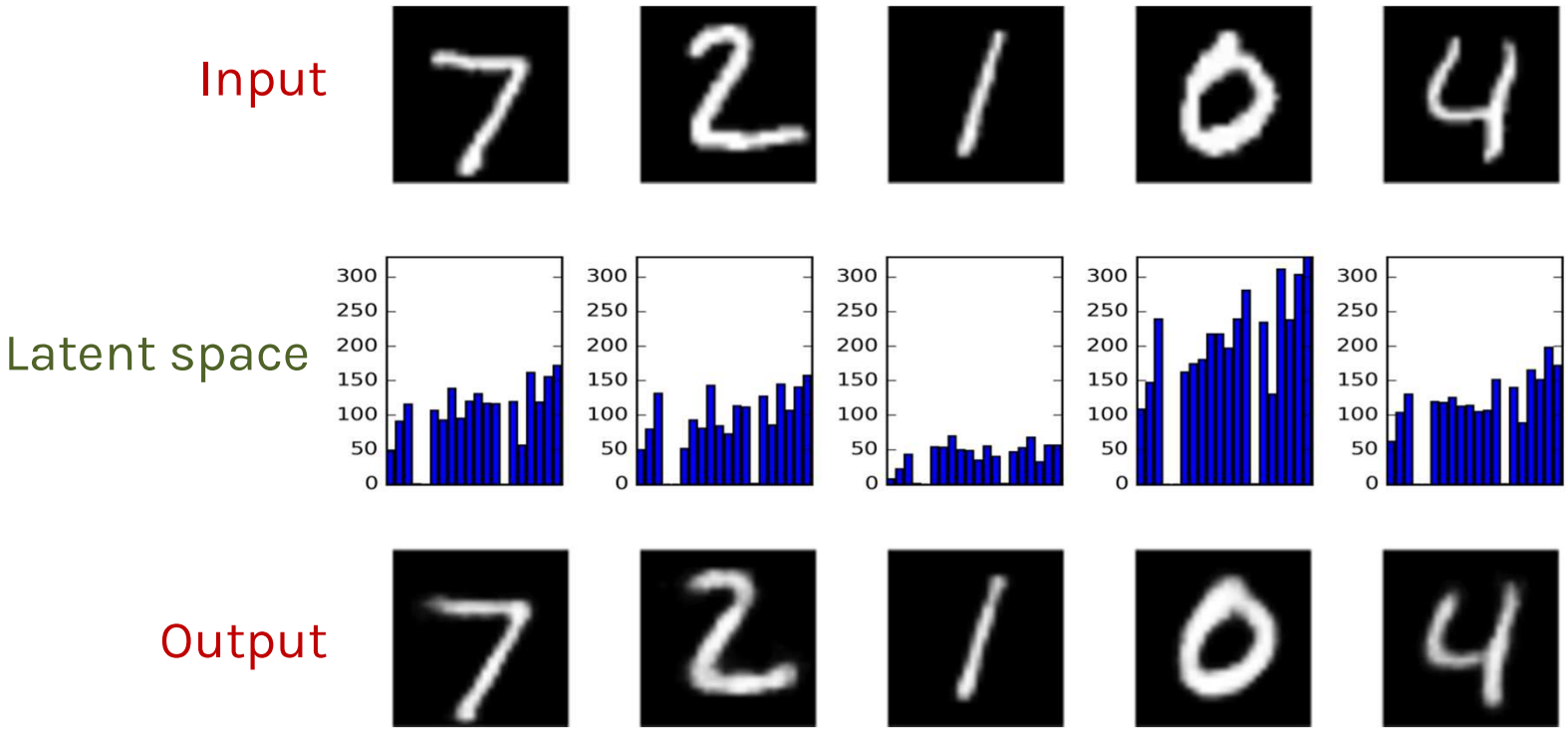


Image taken from A. Glassner, Deep Learning, Vol. 2: From Basics to Practice

Exploring autoencoders

- If latent space does not contain any information, then changing the values randomly won't change the output or it will not be robust.

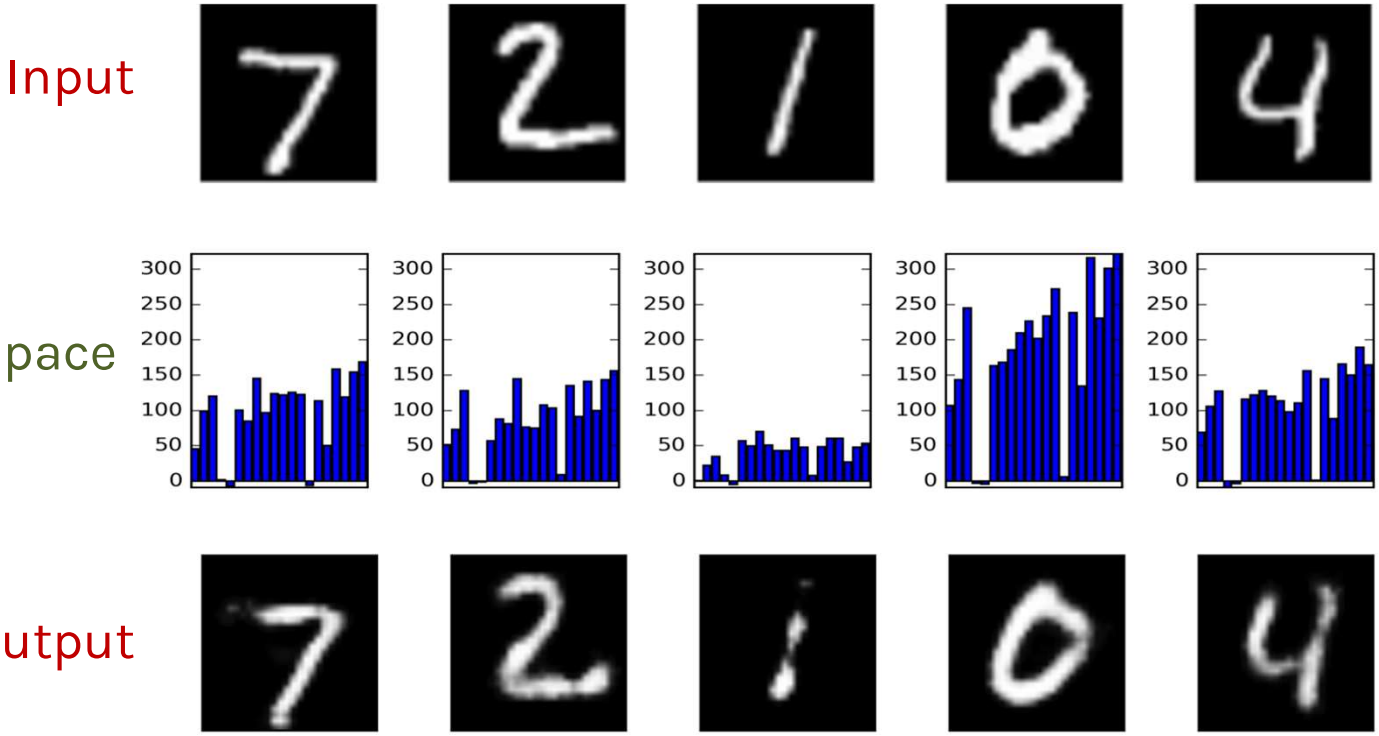


Image taken from A. Glassner, Deep Learning, Vol. 2: From Basics to Practice

Exploring autoencoders

- If latent space does not contain any information, then changing the values randomly won't change the output or it will not be robust.

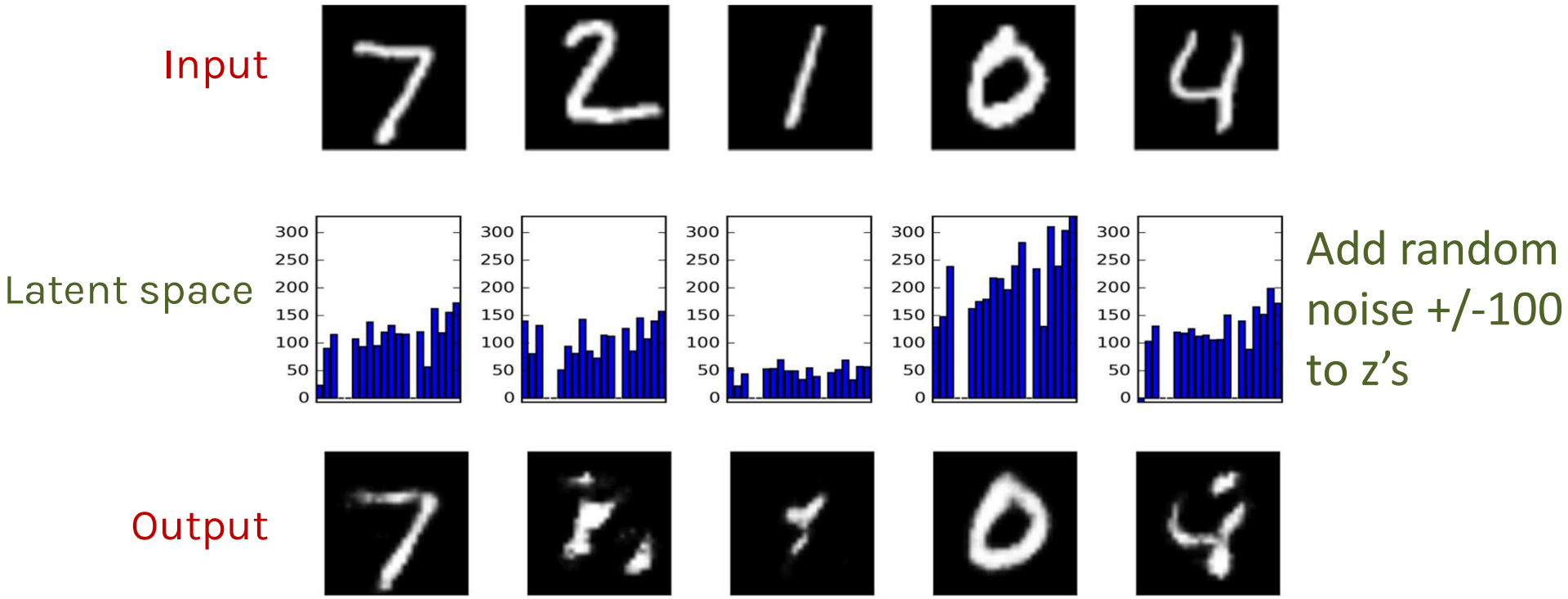


Image taken from A. Glassner, Deep Learning, Vol. 2: From Basics to Practice

Exploring autoencoders

- If latent space does not contain any information, then changing the values randomly won't change the output or it will not be robust.

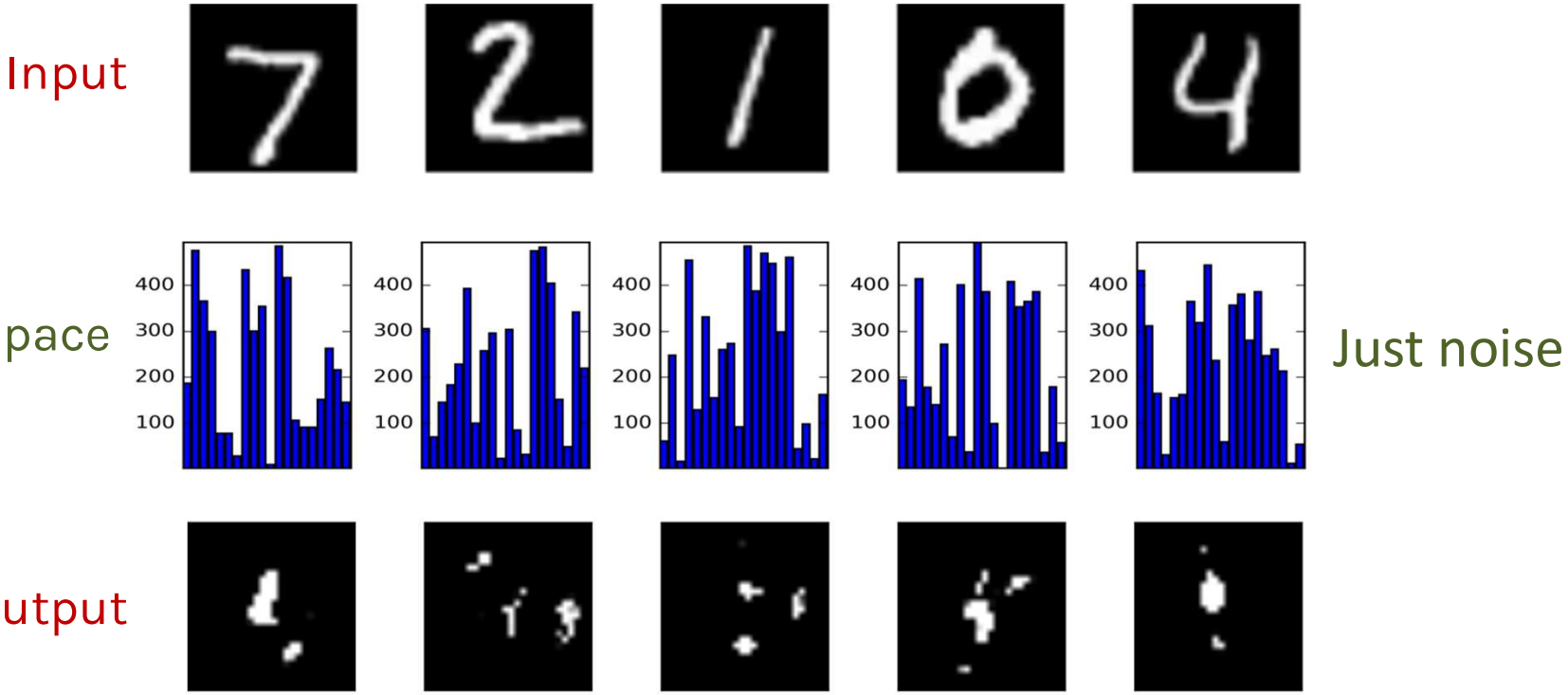
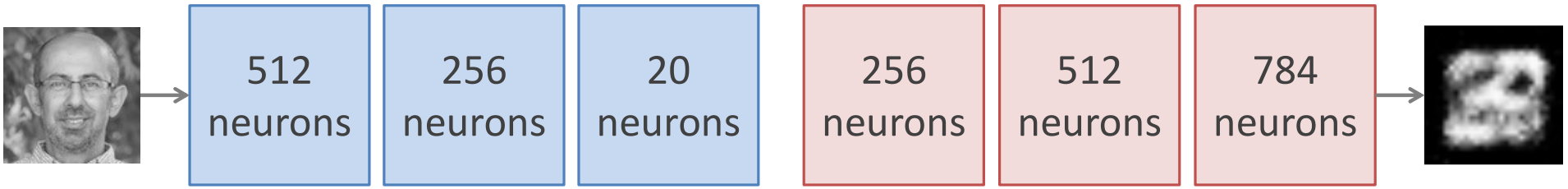


Image taken from A. Glassner, Deep Learning, Vol. 2: From Basics to Practice

Applying to novel input

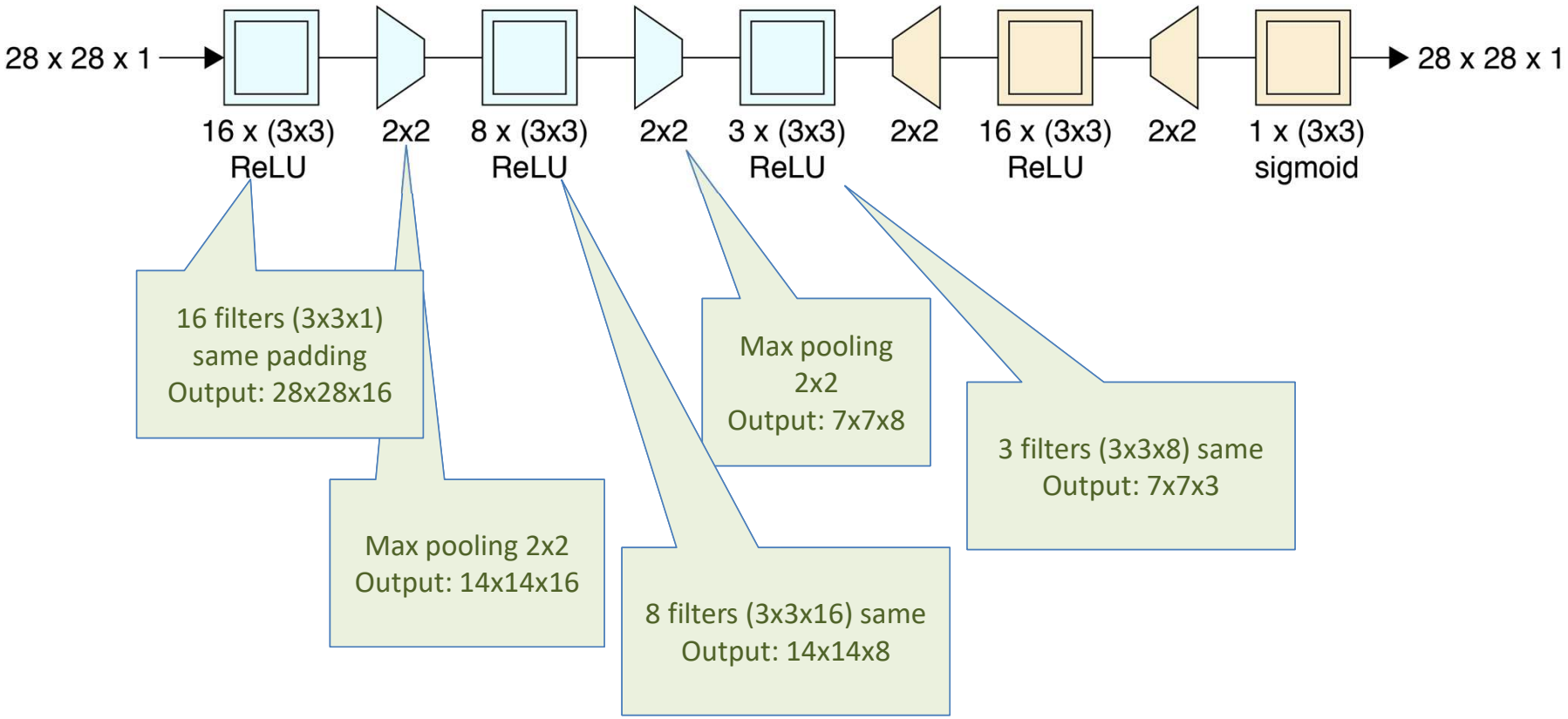
- Check again if the deep AE trained on MNIST works with an image of a face?



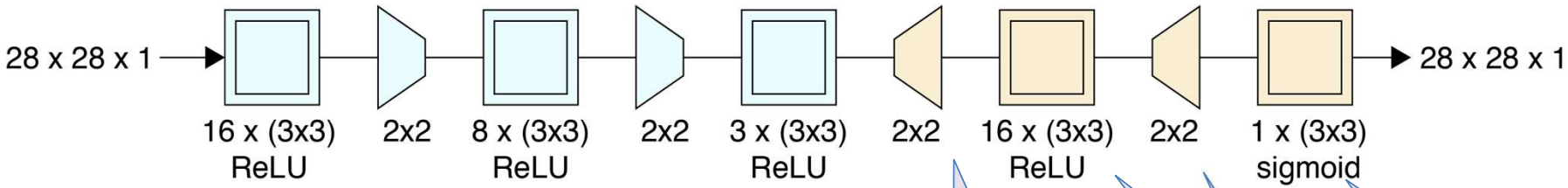
NO

Since we are dealing with images, it is best to use CNNs

Convolutional Autoencoders



Convolutional Autoencoders



3 filters (3x3x8) same
Output: 7x7x3

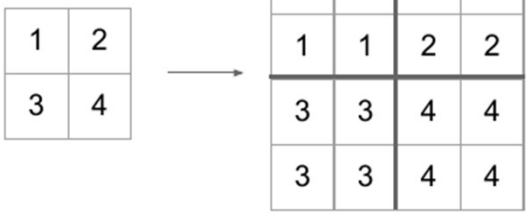
Upsampling 2x2
Output: 14x14x3

16 filters (3x3x3) same
Output: 14x14x16

1 filter (3x3x16) same
Output: 28x28x1

Upsampling 2x2
Output: 28x28x16

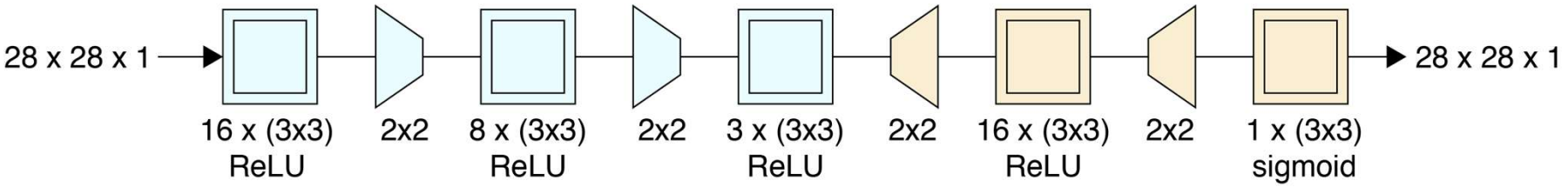
Nearest Neighbor



Input: 2 x 2

Output: 4 x 4

Convolutional Autoencoders



Original Images



Reconstructed Images with Conv AE



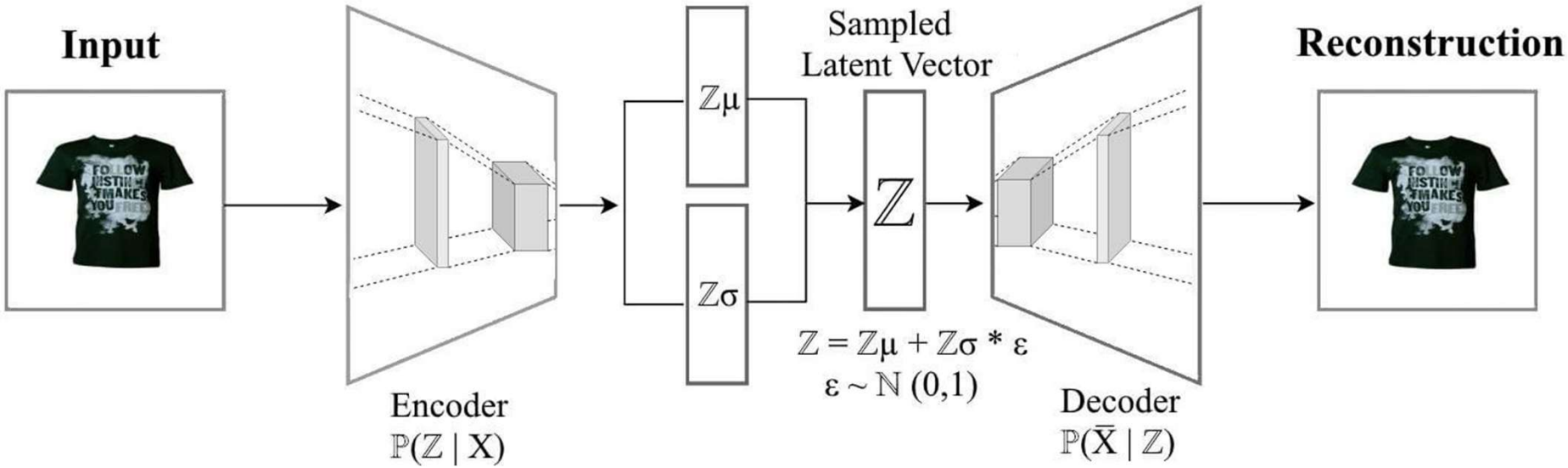
ConvAE latent variables



DeepFCN latent variables

Variational Autoencoders (VAE) – an improvement

- Keeps track of means and standard deviations in the latent space
- VAE preserve details unlike Autoencoders that tend to average things



A history of generative AI models

- 2014-2017: The VAE and GAN era
  - VAE invented in Dec. 2013, followed by GAN in 2014
  - Changes to GAN model architecture (DCGAN, 2015), loss function (Wasserstein GAN, 2017), and training process (ProGAN, 2017)
  - GANs used for image-to-image translation (pix2pix, 2016, and CycleGAN, 2017) and music generation (MuseGAN, 2017)
- 2018-2019: The Transformer Era
  - Transformer rose to prominence with GPT (decoder-only Transformer)
  - Larger language models built with GPT-2 (2018) and T5 (2019)
- 2020- The Big Model Era
  - Merging of ideas from different generative modeling families
  - DDPM and DDIM lay foundations for large image generation models
  - Large language models: GPT-3 (2020), LaMDA (2022), PaLM (2022), OPT (2022), GPT-J (2021), and GPT-NeoX (2022), BLOOM (2022)
  - Multimodal models: DALL.E (OpenAI), DALL.E 2 (2022), Imagen (2022), and Parti (2022)
  - Advancement in Latent Diffusion (2021) and Stable Diffusion model (released by Stability AI)

